Notes for Warehouse-Scale Operating Systems

MIRI-OS

Yuechao Liu
Universitat Politècnica de Catalunya

2026-10-20

Contents

Process Scheduling 8
1 Scheduler Architecture 8
1.1 Monolithic Scheduler 8
1.2 Two-level scheduler 8
1.3 Shared-state Scheduler 9
1.4 Fully Distributed Scheduler 9
1.5 Hybrid Scheduler 9
2 Time-sharing 9
2.1 Single Global Queue 9
2.2 Affinity Scheduling 10
2.3 Local Queues 10
2.4 Load Balancing 11
3 Space-sharing 11
3.1 Partitioning Mechanisms 11
3.2 Variable Partitioning Scheduling Policies 12
3.2.1 First Come First Serve (FCFS) 12
3.3 Backfilling 13
3.3.1 EASY Backfilling (Earliest Arrival Start-time Yield) 13
4 Adaptive Partitioning Scheduling Policies 14
4.1 Equipartition (Adaptive) 14
4.2 Proportional Partitioning 14
5 Dynamic Partitioning Scheduling Policies 15
5.1 Equipartition (Dynamic) 15
5.2 Efficiency Preservation (Hybrid) 16
6 Gang Scheduling 17
6.1 First Fit 18
6.2 Best Fit 19
6.3 Varients 20
6.3.1 Left-Right Packing variant (Lower Job Overlaps) 20
6.3.2 Alternative Scheduling (Schedule a Job Multiple Times if Has Space) 20
6.3.3 Slot unification (Merge Slots) 20
PE Allocation 20
1 Contiguous Partitions 20
1.1 Rectangular Allocation 20
1.2 Contiguous Leak 21
2 Non-contiguous Partitions 21
2.1 Non-contiguous Leak 21
3 Scheduling Metrices 22
3.1 Wait Time 22
3.2 Response Time (Turnaround Time, Flow) 22
3.3 Slowdown (Stretch) 23
3.4 Bounded Slowdown 23
3.5 Per-processor Slowdown 23
3.6 Expansion Factor (XFactor) 23
Process Resilience 23
1 Resilience Metrics 24
1.1 Reliability: Mean Time to Failure (MTTF) 24

1.2 Maintainability: Mean Time to Repair (MTTR) 24
1.3 Availability: Percentage of Time when Available 24
2 Fault Tolerance 24
3 Redundancy 24
3.1 Modular Redundancy 24
3.1.1 Double Modular Redundancy 24
3.1.2 Triple Modular Redundancy 25
3.2 State Machine Replication (SMR) 25
4 Quorum Systems 26
4.1 Crash Tolerance 26
4.2 Crypto Byzantine Tolerance 26
4.3 Verbal Byzantine Tolerance 26
5 Recovery Strategies 27
5.1 Backward Recovery 27
5.2 Global Consistent Recovery State 27
5.2.1 Independent Checkpointing 27
5.2.2 Coordinated Checkpointing 28
5.2.3 Recovery Message Handling 28
5.3 Failure Prediction 28
5.4 Log-based recovery 28
Virtualisation 28
1 Process Virtualisation 29
2 System/Hardware Virtualisation 29
2.1 Full Virtualisation 29
2.1.1 Bare-metal Hypervisor (Type I) 29
2.1.2 Hosted Hypervisor (Type II) 30
2.1.3 Modern Hosted VMMs 30
2.1.4 Sensitive Instructions: Trap-and-Emulate 30
2.1.5 Binary Translation 31
2.1.6 Protection Rings 31
2.1.7 Hardware-Assisted Virtualisation 32
2.1.7.1 VMX mode 32
2.1.7.1.1 VMX Transitions 32
2.1.7.1.2 VM Control Structures (VMCS) 32
2.2 Paravirtualisation 33
2.3 Virtualisation of OS functions 33
2.3.1 VCPU Scheduling (Double Scheduling) 33
2.3.1.1 Borrowed Virtual Time (BVT) 33
2.3.1.1.1 Virtual Time 33
2.3.1.1.2 Context Switch Condition 33
2.3.1.1.3 Starvation Avoidance: Scheduler Virtual Time (SVT) 33
2.3.1.1.4 Time-Critical Support: Effective Virtual Time (EVT) 33
2.3.1.2 Lock Holder Preemption (LHP) problem 34
2.3.1.3 Pause-Loop Exiting (PLE) 34
2.3.1.4 Paravirtualised Spinlocks 34
2.3.1.5 Gang Schedulling 34
2.3.2 Memory Virtualisation 34
2.3.2.1 Software Memory Virtualisation 34
2.3.2.2 Hardware-Assisted Memory Virtualisation 35

2.3.3 I/O Virtualisation 36
2.3.3.1 Hypervisor Direct Mode 36
2.3.3.2 Service-VM/Hosted model 36
2.3.3.3 Pass-Through Model 37
2.3.3.4 I/O Interception in Full Virtualisation 37
2.3.3.5 I/O Interception in Paravirtualisation 38
2.3.3.6 Hardware-Assisted I/O Virtualisation 39
2.4 OS-Level Virtualisation (Containers) 39
2.5 Application-Level Virtualisation (Programming-Environment Virtualisation) 41
3 Virtual Clusters and Resource Management 41
3.1 VM Deployment 41
3.2 Live VM Migration 41
3.2.1 Memory Migration 41
3.2.1.1 Pre-Copy 41
3.2.1.2 Post-Copy 42
3.2.2 Storage Migration 43
3.2.2.1 Shared-Nothing: Snapshotting 43
3.2.2.2 Shared-Nothing: Dirty Block Tracking 43
3.2.2.3 Shared-Nothing: I/O Mirroring 43
3.2.3 Network Migration 43
3.3 VM Allocation 44
3.4 VM Consolidation 44
3.5 VM Load-Balancing 44
3.6 VM Elasticity 44
3.7 VM Checkpoint/Recovery 44
3.8 VM Replica Failover 44
In-Memory Data Management 44
1 Desktop 44
2 Pre-WSC 45
2.1 Distributed Address Space 45
2.2 Shared Address Space 45
2.2.1 Replicated Address Space 45
2.2.2 Aggregation of Address Space 46
3 WSC 46
3.1 CAP Theorem 46
3.2 Not-Only-CAP: PACELC 46
3.3 Eventual Consistency 47
3.4 Session Guarantee 47
3.5 Data-Centric: Consistent Prefix 47
3.6 Data-Centric: Bounded Staleness 47
3.7 User-Centric: Monotonic Read 47
3.8 User-Centric: Read-Your-Writes 48
4 Consistency Protocols 48
4.1 Process Groups 48
4.1.1 Hierarchical Process Groups (Primary-Based Protocols) 48
4.1.2 Flat Process Group (Replicated Protocols) 48
4.2 Primary-Based Protocol 48
4.2.1 Primary-Backup Remote-Write Protocol 48
4.2.2 Primary-Backup Local-Write Protocol 49

4.3 Replicated-Write Protocol 50
4.3.1 Active Replication 50
4.3.2 Quorum-Based Protocols 51
5 Redis 51
5.1 Consistency Protocol 51
5.2 Fault Tolerance 51
5.3 Commands 52
On-Disk Data Management 52
1 File System Properties 52
2 Naming 52
3 Operations 52
4 Distributed File Systems 53
4.1 Semantics for Shared Files in Distributed Systems 53
4.1.1 Sequential Semantics (UNIX) 53
4.1.2 Session Semantics 53
4.1.3 Immutable Files 53
4.1.4 Transactions 53
4.2 Architectures 53
4.2.1 Client-Server Architecture 53
4.2.1.1 Implementation 54
4.2.1.2 Client Side 54
4.2.1.3 Server Side 54
4.2.1.4 Access Models 55
4.2.1.4.1 Remote Access Model 55
4.2.1.4.2 Upload/Download Model 55
4.2.1.5 Server Stateness 55
4.2.1.5.1 Stateful 55
4.2.1.5.2 Stateless 55
4.2.2 Cluster-Based Architecture 55
4.2.2.1 Replication 55
4.2.2.2 Distribution (Partition) 55
4.2.2.2.1 Consistency Problems 56
4.3 IBM General Purpose File System (GPFS) 56
4.4 GFS 56
4.4.1 Interface 57
4.4.2 Cluster 57
4.4.3 Single Master 57
4.4.4 Chunk Servers 57
4.4.5 Read Path 58
4.4.6 Write Path 58
4.5 HDFS 58
5 Distributed Databases 58
5.1 Scale Up/Out 59
5.2 Key-Value (NoSQL) Data Model 59
5.3 Storing Data 60
5.3.1 Block Storage Device 60
5.3.1.1 File Creation 60
5.3.1.2 File Access 60
5.3.2 Object Storage Device (OSD) 61

5.3.2.1 File Creation 61
5.3.2.2 File Access 61
5.4 Cassandra 62
5.4.1 Replication 62
5.4.2 Consistency 63
5.4.3 Consistent Hashing 63
5.4.4 Scalability 64
5.4.4.1 Gossip Protocol 64
5.4.4.2 Bootstrap (Add Node) 64
5.4.4.3 Removing Nodes 65
5.4.5 Write Path 65
5.5 Read Path 66
5.5.1 Key-Space 67
5.5.2 Tables 67
Energy Management 67
1 Metrics 68
1.1 Power Usage Efficiency (PUE) 68
1.2 Server Power Usage Efficiency (SPUE) 68
1.3 Total Power Usage Efficiency (TPUE) 68
1.4 Energy Efficiency of WSCs 68
1.5 Data Centre Energy Productivity (DCeP) 69
2 Power Metering 69
2.1 Integrated Metering on Hardware 69
2.2 External Meter 69
3 Power Modeling 70
4 Efficiency of IT Equipment 70
4.1 Low-Power Modes 70
4.1.1 Inactive Low-Power Modes 70
4.1.2 Active Low-Power Modes 71
4.1.3 Non-Active and Active Power Saving Mode 72
4.1.4 Default and Active Power Saving Mode 73
5 Resource Allocation and Scheduling 73
5.1 VM Consolidation 74
5.2 Live VM Migration 74
5.3 VM Allocation 75
6 WSCs Ecosystems 75
7 Cooling Subsystem 75
7.1 Free Cooling 76
7.2 Computer Room Air Conditioning (CRAC) Based Cooling 76
7.2.1 Insufficient air flow 77
7.3 High-Density Cooling 77
7.3.1 In-Rack Cooling 78
7.3.2 In-Row Cooling 78
7.3.3 Cold Plates 79
7.4 Efficiency of Cooling Subsystem 79
7.4.1 Hardware Perspective 79
7.4.2 Software Perspective 79
7.4.2.1 Workload Adapted Cooling 79
7.4.2.2 Temperature-Aware Allocation 80

8 Power Supply Subsystem 80
8.1 Uninterruptible Power Supply (UPS) 80
8.1.1 Active Power Input Switch 80
8.1.2 Energy Storage 80
8.1.3 Remove Voltage Spikes 80
8.1.3.1 Traditional Double Conversion 81
8.1.3.2 DC System 81
8.2 Power Distribution Unit (PDU) 82
8.3 Renewable Energy 82
8.3.1 Workload Shaping 82
8.3.2 Efficiency of Energy Usage 82
8.3.3 On-site Renewable Generation 82

Process Scheduling

Two dimensions of scheduling on WSC OS:

- **Which** process to run next
- **Where** must the process run
  - On which **Processing Element (PE)**: which Computer, which CPU

Two Scheduling Approaches

- Time-sharing: Processes run on the **same PE** by doing recurrent context switching
- Space-sharing: PEs are partitioned among multiple processes that execute side-by-side

Job

- A job is an application in execution, consists of **multiple** concurrent **processes/threads**.
- it is the **unit of scheduling** defined by
  - **Arrival time**
  - **Number of required PEs** (i.e. number of threads/processes), aka Job width, Job size
  - **Runtime estimation**, aka Job Length
- Classification
  - Number of processes
    - Serial
    - Parallel
  - Interaction with the user
    - Interactive: Immediate response, I/O redirected to a terminal. require **fast turnaround time**
    - Batch: Submitted to a queue to be scheduled, run when resources available
  - Capacity to adjust the number of processes/threads
    - Rigid: #PEs **predefined**
    - Moldable: #PEs determined **at start time**, cannot change after
    - Evolving: #PEs change **at runtime** on its **own initiative**, request scheduler for PEs
    - Malleable: #PEs change **at runtime** according to **availability of PEs** from the scheduler

1 Scheduler Architecture

1.1 Monolithic Scheduler

Handles **all the jobs** using the **same logic**

- ↑ **Up-to-date global view** of the whole cluster
  - better scheduling decisions
  - easy to enforcing constraints
- ↓ Slow scheduling
- ↓ hard to handle **mixed workloads**
  - interactive jobs
  - Queuing effects (head-of-line blocking)

1.2 Two-level scheduler

A **central resource manager** partitions cluster dynamically among **multiple independent** workload-specific **schedulers**. (Each schedulers make actual task placement)

- ↑ Placement tailored for specific workloads
- ↑ Better scalability (mitigate load on the scheduler)
- ↓ Limited cluster view
  - Hard to enforce global constraints

- Hard to detect interference

1.3 Shared-state Scheduler

Multiple workload-specific schedulers have access to the **entire cluster**

Cluster state is **replicated** and can be independently updated by schedulers using **optimistic concurrency control** (abort/retry update if there are conflicting changes)

- ↑ Fast scheduling, good scalability
- ↑ Placement logic can be tailored to specific workloads
- ↓ Needs to decide with (occasionally) stale information (update is visible only after commit, update propagation takes time)
- ↓ Degraded performance under high contention

1.4 Fully Distributed Scheduler

Multiple independent schedulers serve incoming workloads **without any coordination** between them.
Use only their **local**, **partial** and often **outdated view** of the cluster

- Jobs are submitted to **any** scheduler
- ↑ Fast scheduling, high throughput, high scalability
- ↓ Bad scheduling decisions, hard to enforce global constraints, hard to detect interference
  - More suitable for **short-running** jobs where the effect of bad decisions don't accumulate overtime

1.5 Hybrid Scheduler

Combines **fully-distributed** scheduler with **Monolithic** scheduler

Two Scheduling Paths

- Short jobs: Fully-distributed scheduler
- Long jobs: Centralised scheduler

↑ Useful for heterogeneous workloads

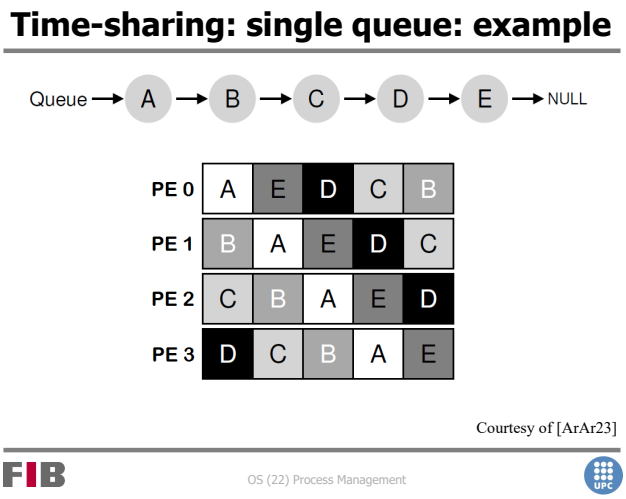
2 Time-sharing

- Multiple processes run on the **same PE** by doing recurrent context switching
- Processes can be **preempted** by others during execution and restart later
- ↑ Increase utilisation of PEs
- ↓ Overhead of context switches

2.1 Single Global Queue

Put all jobs that need to be schedules into a single global queue

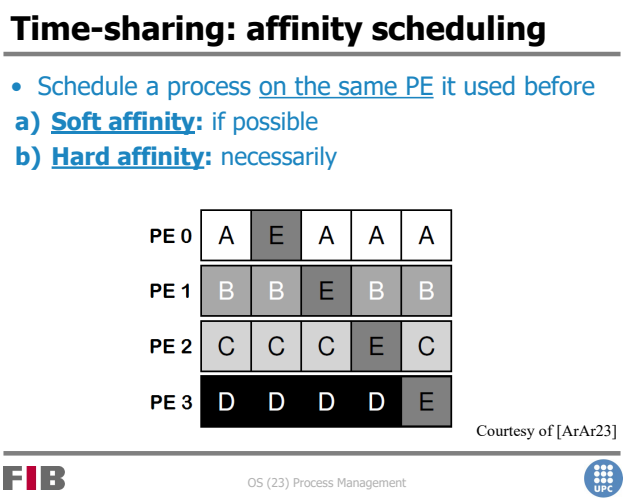
- ↑ Simplicity
- ↑ Automatic load balance: workload distributed across all PEs by construction
- ↓ Does NOT preserve **data locality**
- ↓ Hard to implemented (distributed) global queue (without shared memory)
- ↓ Synchronisation needed, low scalability



2.2 Affinity Scheduling

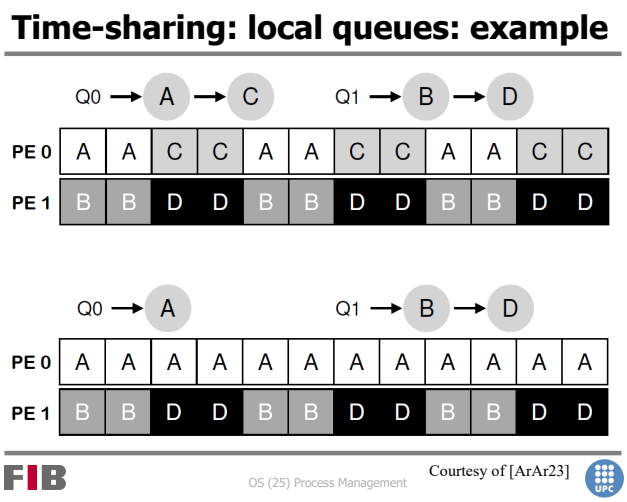
Scheduling process **on the same PE** it used before

- Soft affinity: is possible
- Hard affinity: necessary



2.3 Local Queues

Independent queues for each PE



2.4 Load Balancing

Balance by migrating active processes between PEs
Types

- Receiver-initiated (Work stealing): idle PEs steal process from overloaded PEs
- Sender-initiated (Work pushing): overloaded PEs offload processes to underloaded or idle PEs

↑: Better load distribution ↓ Communication and processing overhead ↓ Does not preserve data locality

3 Space-sharing

PEs are partitioned among multiple processes that execute side-by-side

- Normally, one process per PE, but we could combine time-sharing to schedule multiple processes a PE
- Jobs run side-by-side until completion

Advantages and disadvantages

- ↑ Partition is dedicated to a job
- ↑ Avoids context switch
- ↓ Interconnects and I/Os may be shared by several jobs
- ↓ Poor utilisation: wasted when a process **block**s

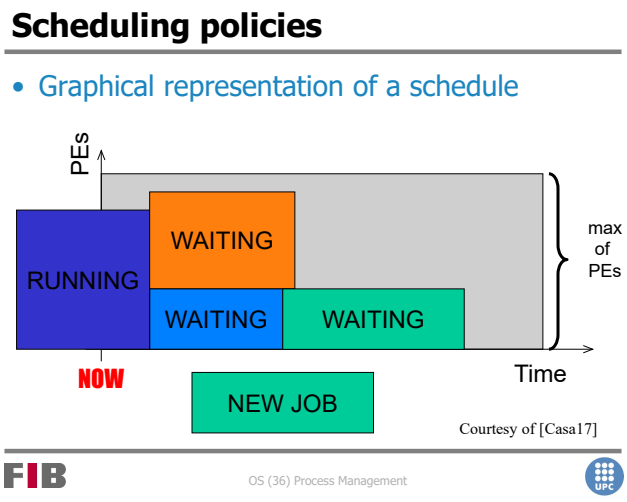
3.1 Partitioning Mechanisms

- **Fixed Partitioning**
  - Partitions are set by the administrator
    - typically by using queues for different sizes and limits (#PEs, execution time etc)
  - Partitions can be modified **only by rebooting**
  - ↑ Simple, preserves data locality
  - ↓ Arbitrary queuing
  - ↓ Internal fragmentation (unused PEs within a partition)
- **Variable Partitioning**
  - Partition size is set at **submission time** (of jobs)
  - Partition size is based on incoming **job request** (specified by the job)
  - Partition size does not change during job runtime

  - ↑ Matching job request, preserves data locality
  - ↓ Arbitrary queuing
  - ↓ External fragmentation (PEs not allocated to any partition)
  - ↓ Internal fragmentation **if** the architecture imposes restrictions to the partition size
- **Adaptive Partitioning**
  - **Requirement**: **Moldable Jobs** (determine # of processes at **start time**)
  - Partition size is set when the **job starts**
  - Partition size is **based on system load** (specified by the system, taking into account job request)
  - Partition size does not change during job execution
- **Dynamic Partitioning**
  - **Requirement**: **Mellable Jobs** (change PEs **at runtime** according to the availability of PEs in the system)
  - Partition size can change **during job execution** to respond to load changes

3.2 Variable Partitioning Scheduling Policies

- Jobs are queued **until sufficient PEs** become available
- Alternatively, Combine with time-sharing to schedule multiple jobs within a partition



3.2.1 First Come First Serve (FCFS)

- Activation time: Job arrival and termination
- ↑ Fiarness
- ↓ External fragmentation: first job blocks the rest of the queue

Algorithm 1: FCFS()

```
1 job ← Queue.head
2 if (availablePEs ≥ job.requiredPEs then
3 |  Start(Queue, job);
4 else
5 |  Wait;
6 end
```

3.3 Backfilling

Backfill jobs that **arrive later** to the head of the queue (**fill holes**)

- Reduce fragmentation of **breaking FCFS order**
- Restriction: Backfilled jobs **cannot** delay reserved jobs (defines N reserved jobs)
- Activation time: Job arrival and termination
- Requires job runtime estimation and required PEs

3.3.1 EASY Backfilling (Earliest Arrival Start-time Yield)

- Only **ONE** reserved jobs
  - Guarantees the first job does not get delayed, but the second job may get delayed
- **Shadow time**: expected **start time** of the first job (in the queue)
  - In exams, this would mean that the first schedule-able time of the job (because the user didn't specify it)
- **Extra PEs**: Available PEs when the first queued job starts
- Notes
  - Limited Vision
    - On each invocation, it only make decisions for **the current time**
    - The algorithm is invoked **whenever a job finishes** or **arrives**
  - Runtime Estimation
    - typically, jobs are killed if it takes longer than estimated (avoid delaying other jobs)
    - Aggressive Estimation: goes through the queue faster but may be killed
    - Conservative Estimation: Harder to backfill, but will less likely to be killed

Algorithm 2: CanSchedule(S: Schedule, J: Job)

return S.currentAvailablePEs $\geq$ J.requiredPEs

Algorithm 3: EASY-Backfilling(S: Schedule)

```
while CanSchedule(S, Q[0])
    Q.remove(Q[0]);
    Schedule(S, Q[0]);
J_r ← Q[0];
for i = 1..|Q|
    J_i ← Q[i];
    if (t + Runtime(J_i) ≤ ShadowTime(J_r, S)) ∨ (J_i.NeededPEs ≤ J_r.ExtraPEs)
        if CanSchedule(S, J_i)
            Q.remove(J_i)
            J_r.ExtraPEs ← J_r.ExtraPEs - J_i.NeededPEs;
            Schedule(S, J_i)
```

Variations

- Order of queues
  - FCFS: EASY backfilling
  - Priorities: Slack-based backfilling
  - Shortest-estimated-runntime-first: SJF backfilling
- Number of reserved jobs

  - One: EASY backfilling
  - ALL: conservative backfilling
    - HOW DO WE RESERVE ALL JOBS AND CALCULATE THEIR SHADOW TIME?
    - Idea: Backfilled jobs **cannot** delay **ANY** jobs
  - adaptive: selective backfilling
    - only reserve jobs whose expected slowdown exceeds a threshold
- Look-ahead in the queue (#jobs analysed simultaneously to backfill)
  - One job at a time
  - the whole queue at once (HOW? some policices?)

4 Adaptive Partitioning Scheduling Policies

4.1 Equipartition (Adaptive)

Aim for equal-sized partitions for all jobs
Activation time: job arrival/termination

Consider only **waiting jobs**
- Allocate **one PE** at a time for each waiting job
- **Run the job** when the allocated **PEs** reach its requirement
  - Then start over again with the new **waiting jobs** (excluding this job because it's scheduled)

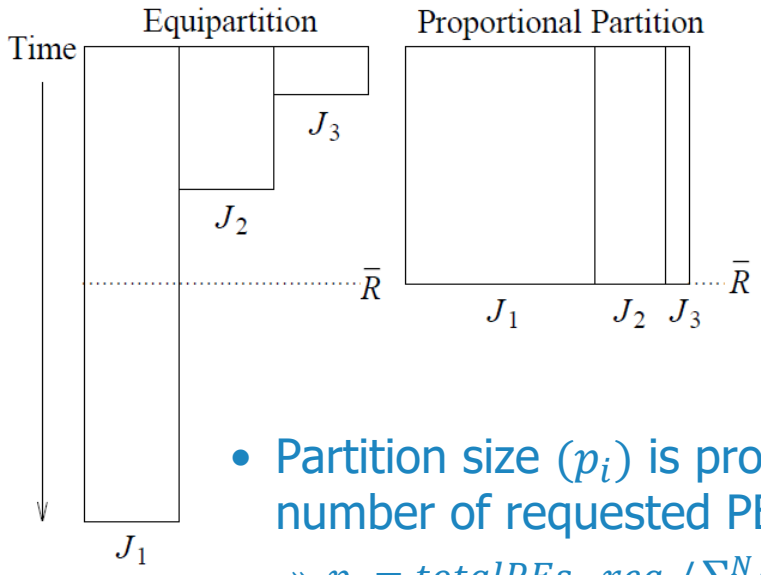
4.2 Proportional Partitioning

Aim for PEs proportional to the request

$p_i = \text{totalPEs} \cdot \frac{\text{req}_i}{\sum_{j=1}^{N}(\text{req}_j)}$ (Do we round down or up?)

The following figure show the difference between **Equipartition** and **Proportional Partitioning**

Policies: proportional partitioning



policy	job	req_i	p_i	rt_i
EQUI	J_1	1000	20	50
	J_2	400	20	20
	J_3	100	20	5
PROP	J_1	1000	40	25
	J_2	400	16	25
	J_3	100	4	25

number of PEs = 60

- Partition size (p_i) is proportional to the number of requested PEs (req_i)
  - » $p_i = totalPEs \cdot req_i / \sum_1^N req_j$

FIB OS (50) Process Management Courtesy of [Brec94] UPC

5 Dynamic Partitioning Scheduling Policies

5.1 Equipartition (Dynamic)

Aim for equal-sized partition for all jobs **Activation time**: job arrival/termination, job requirement change

Consider **all jobs**

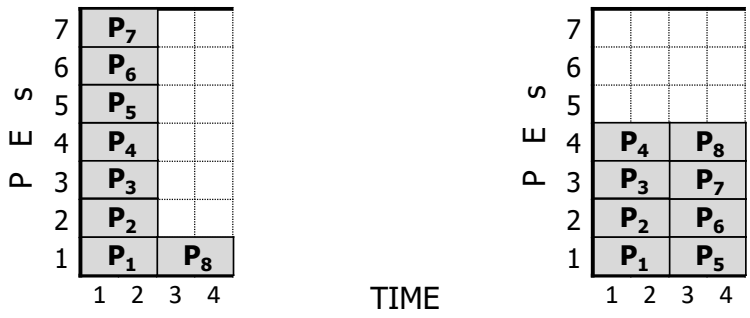
- Allocate **one PE** at a time for each job
- **Run the job** when the allocated **PEs** reach its requirement

Some PEs might be preempted from running jobs and given to other jobs
PEs of a finished job are **equally** divided among ALL the other jobs

5.2 Efficiency Preservation (Hybrid)

Policies: efficiency preservation

- Consider runtime parameters such as the achieved speed-up or the efficiency of the jobs to decide the partition size
  - e.g. running a job with 8 processes on 7 PEs does not provide any benefits over using only 4 PEs



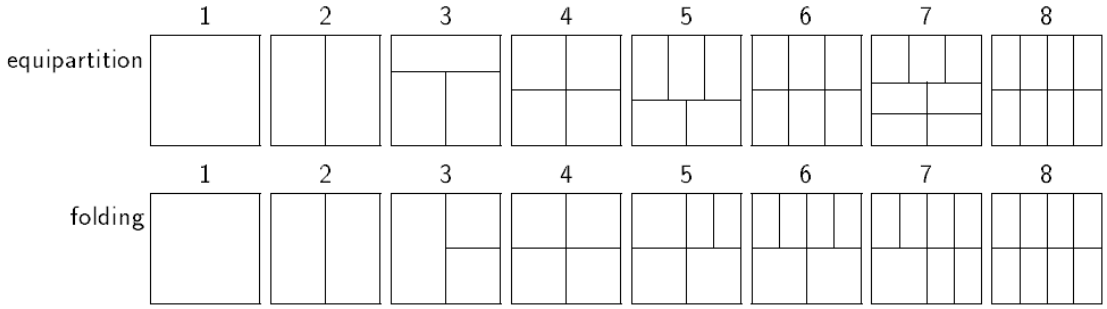
Folding policy

- On job arrival
- **the largest** current partition is **folded in half** (half of the PEsare freed)
  - assume #PEs = 2^n
- Processes on each partition will **time-share** PEs

In the following figure, the x and y axis are combined to show spacial distribution of the PEs in a mash

Policies: efficiency preservation

- **Folding** policy for efficiency preservation
  - On job arrival, the largest current partition is folded in half, freeing PEs for the new job
    - Assumes that total number of PEs is a power of two, and jobs are designed to execute optimally with all the PEs
  - Processes on each partition will time-share the PEs



FIB OS (54) Process Management Courtesy of [Feit97] UPC

6 Gang Scheduling

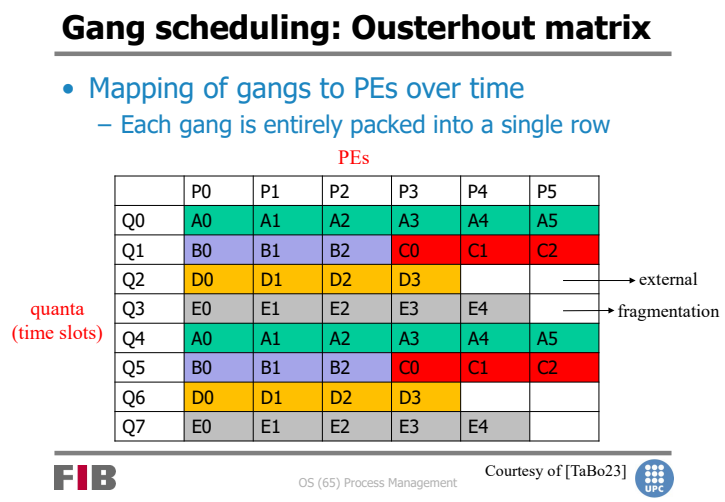
Schedule all **processes** of **a parallel job** together (a scheduling unit)

- Processes in a job are grouped into a gang
- Processes in a gang execute **simultaneously** in **distinct PEs** (using a one-to-one mapping)
- Time-sharing between gangs, **all** processes within the same gang must be preempted and rescheduled at the same time
- ↑ Reduces intereactions between processes within the same gang (P_1 send messages to P_2)
- ↓ Overhead for context switch and coordination
- ↓ May have external fragmentation

Assignment

- A job is usually assigned to **contiguous PEs** to facilitate **slot unification** when job terminates
- A job is usually assigned to the same PEs to exploit data locality (affinity scheduling)

Time Slot (in a Ousterhout Matrix)
a quanta (time slot) consists a row of available PEs at each time



6.1 First Fit

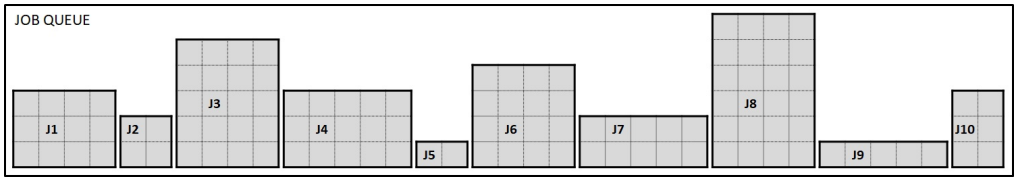
- Used slots are scanned in sequential order
  - From $t = 1, 2, 3...$ and at each time, from $\text{PE} = 1, 2, ...$
- The **first one** with sufficient capacity is chosen
- If current time slot (at t) has enough capacity, open **a new time slot** (at $t + 1$)

In the following example, we can see that

- We choose to start from PE=8 when the current time slots of our job is even and start from PE=1 when it is odd
  - $t = 1, 3, 5, ...$ is considered even and $t = 2, 4, 6, ...$ is considered odd
- We schedule jobs in order $(J_1, J_2, J_3, ...)$
  - When we want to schedule J_3 at $t = 1$, we cannot find available slots, so we open a new time slot at $t = 2$ and schedule J_3, since J_3 needs 4 time slots, it is schedule starting at PE=8
  - When we want to schedule J_4, we find it can be schedule at $t = 1$ (sequential scan)

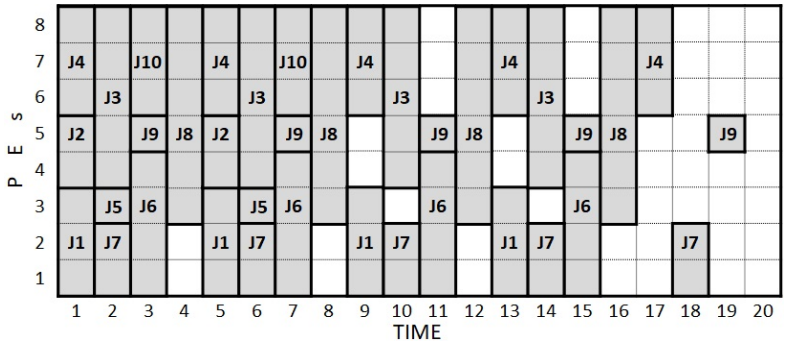
Name and Surname:

2. (1.25 pts) Some jobs are currently waiting in the queue to be scheduled in a WSC with 8 PEs. Each job shows graphically the number of requested PEs (vertical axis) and its duration (horizontal axis).

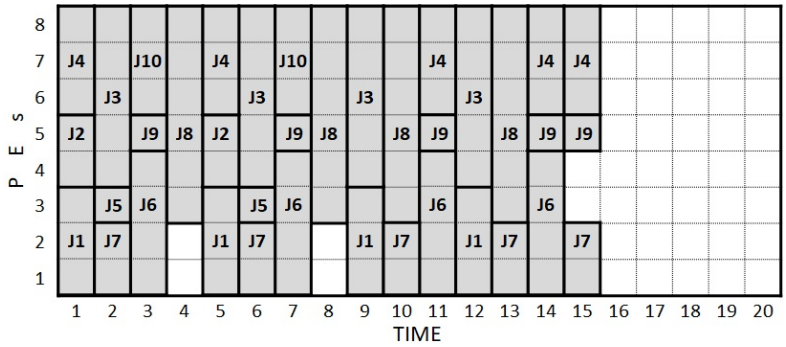


a) Indicate how the jobs are scheduled by using *Gang Scheduling*, which is configured with a quantum of one time unit, uses 'Left-Right First Fit' to allocate jobs to slots (jobs in the queue are scanned sequentially; free slots are scanned sequentially and the first one with sufficient capacity to run the job is chosen; if no free slot has sufficient capacity, a new slot is used; odd slots are filled starting from PE=1; even slots are filled starting from PE=8), and implements *Affinity Scheduling* (the PEs allocated to a job are the same during its entire execution), if:

i) There is not any re-packing of the jobs upon every job termination.



ii) There is a re-packing of all the jobs upon every job termination. Slots are unified as much as possible, but maintaining jobs affinities.



b) Calculate the *slowdown* for the last job to finish in previous scenario i), and its corresponding *slowdown* in scenario ii).

Solution: Slowdown J9 scenario i) $= (T_W + T_R)/T_R = (14 + 5)/5 = 3.8$
Slowdown J9 scenario ii) $= (T_W + T_R)/T_R = (10 + 5)/5 = 3$

6.2 Best Fit

- Used slots are sorted according to their free capacity (time slots, with multiple space(PE) "cells")

- One time slot with the smallest sufficient capability is chosen
- If none has enough space, a new time slot is created

6.3 Varients

6.3.1 Left-Right Packing variant (Lower Job Overlaps)

Used to lower overlaps between jobs

- start using PEs from left/right
  - In the exam (Jan 2021), we consider the index of TIME slots and even slots starts from PE=1 and odd slots starts from PE=8
  - $t = 1, 3, 5, \ldots$ is considered even and $t = 2, 4, 6, \ldots$ is considered odd
  - It is NOT the duration of the job, but the current time slot t in the scheduling

6.3.2 Alternative Scheduling (Schedule a Job Multiple Times if Has Space)

Schedule a job in multiple slots if corresponding PEs are free

- ↑ Reduce fragmentation
- ↓ Unfair: some jobs may have more opportunity to run in the same period (and may terminate sooner)

6.3.3 Slot unification (Merge Slots)

Merge slots in which jobs use complementary PEs

- ↑ Reduces fragmentation
- ↓ Increases run fraction of all jobs

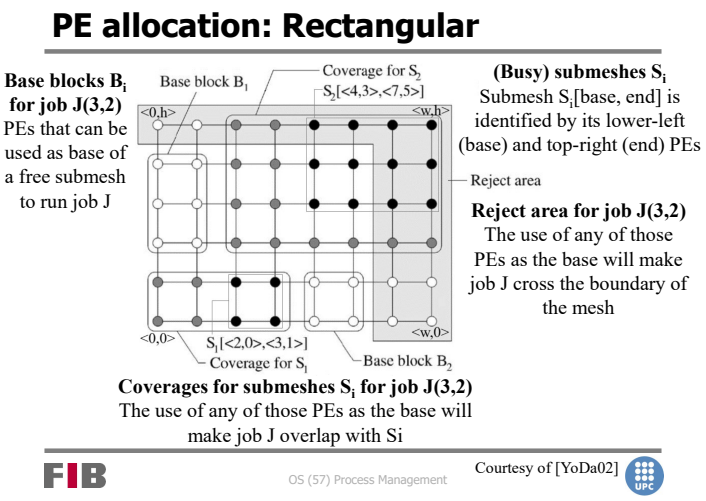
PE Allocation

Select a specific subset of PEs to use

1 Contiguous Partitions

1.1 Rectangular Allocation

We are choosing the bottom left point as the anchor point, the reject area, and coverages are meant to use this bottom left point.

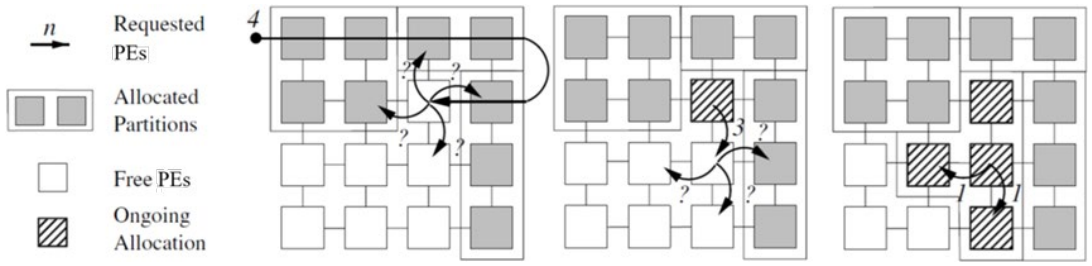


1.2 Contiguous Leak

- Search for free PE ti act as origin using a "snake" wave
- Origin found: test all direct neighbours (↑, →, ↓, ←), add free neighbours to the partition
- Allocate recursively until needed PEs are allocated
- Origin cannot allocate enough PEs: try **next** free PE in the search wave as origin
- Fail when no more PEs are available

PE allocation: Form-free: Leak

- Search for a free PE to act as origin using a 'snake' wave
- Test all direct neighbors of the origin PE to check if any are free ⇒ Allocate each free neighbor to the partition
- Continue recursively until all the needed PEs are allocated
- If no suitable partition can be formed from this origin PE, try next free PE in the search wave as origin
- Allocation fails if no more PEs are available to try



FIB OS (59) Process Management Courtesy of [RoHL07] UPC

2 Non-contiguous Partitions

- ↑ Reduced external fragmentation
- ↑ Better compatibility with dynamic partitioning
- ↓ Overhead due to increased latency and network connection

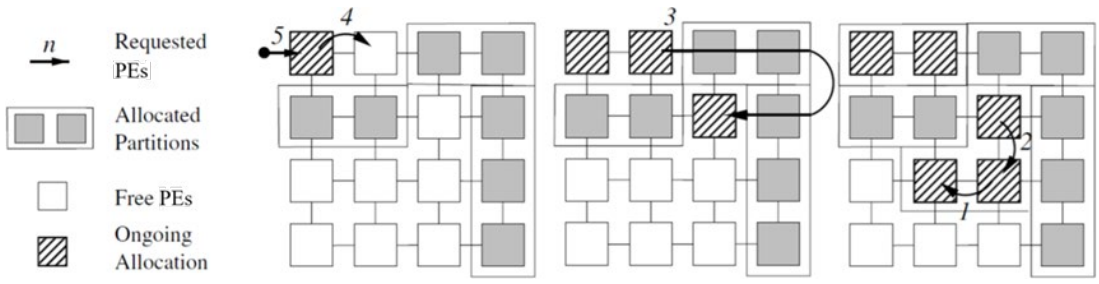
2.1 Non-contiguous Leak

Similar to Contiguous leak, but

- When a source cannot allocate enough PEs, we don't retry by finding a new origin, we just keep finding another origin while keeping the allocated PEs

PE allocation: Non-contiguous: Leak

- Same as the contiguous version, except that a partial allocation insufficient to fulfill the request is kept allocated while the search wave continues from the next origin PE looking for the additional PEs needed



3 Scheduling Metrices

Used to quantify how good a schedule is

3.1 Wait Time

Time spent in the queue waiting to be schedule T_W

- Does NOT capture users' notion of responsiveness
  - Users expect jobs to take time that is **proportional to their length**
  - e.g. a 100s wait is acceptable to a 1s job
  - $\rightarrow$ needs a term for the running running time of the job

3.2 Response Time (Turnaround Time, Flow)

Time elapsed from job **submission** until its **completion** (waiting time + running time)

$$T_{\text{response}} = \underbrace{T_W}_{\text{Waiting Time}} + \underbrace{T_R}_{\text{Runtime}} \tag{1}$$

- Places greater emphasis on longer jobs
- Not effected to a short job waiting a few minutes (as long as $T_W + T_R$ is good)

3.3 Slowdown (Stretch)

$$
\begin{aligned}
T_{\text{slowdown}} &= \frac{\overbrace{(T_W + T_R)}^{\text{Response Time}}}{\underbrace{T_R}_{\text{Runtime, Normalisation Facotr}}} \\
&= \frac{T_W}{T_R} + 1
\end{aligned}
\tag{2}
$$

- Quantifies loss of performance due to competition for PEs
- Captures the responsiveness better
- **Overemphasis** on **very short jobs** (the denominator is small hence the whole ratio is large)
  - **Bounded slowdown** Reduce the influence of short jobs
- Almost equivalent jobs may have very different slowdown
  - Almost Equivalent Jobs: SAME amount of work and SAME response time
  - $(J_1 : T_W = 0s, T_R = 100s, N_{\text{PEs}} = 1), (J_2 : T_W = 90s, T_R = 10s, N_{\text{PEs}} = 10)$
    - $T_{\text{response}} = T_W + T_R = 100s$
    - $\text{Work} = T_R \cdot N_{\text{PEs}} = 100$
    - $(T_{\text{slowdown}})_1 = \frac{T_{\text{response}}}{T_{\text{R1}}} = 1$
    - $(T_{\text{slowdown}})_2 = \frac{T_{\text{response}}}{T_{\text{R2}}} = 10$
  - $\Rightarrow$ **Per-processor slowdown**:
    - $\frac{T_W + T_R}{T_R \cdot N_{\text{PEs}}}$

3.4 Bounded Slowdown

Measure the slowdown relative to some τ to reduce the influence of short jobs.

$$
T = \max\left(\frac{T_W + T_R}{\max(T_R, \tau)}, 1\right) \tag{3}
$$

3.5 Per-processor Slowdown

Divide the slowdown by the number of PEs used (not the numerator's Runtime though).

$$
T = \frac{T_W + T_R}{T_R \cdot N_{\text{PEs}}} = \frac{T_{\text{slowdown}}}{N_{\text{PEs}}} \tag{4}
$$

3.6 Expansion Factor (XFactor)

Pre-execution metric for job schedling, similar to the "Slowdown" but the duration is used as user-specified parameter $\text{XFactor} = \frac{\text{wait + duration}}{\text{duration}}$

Here, at time t_x, the new coming jobs always have $\text{XFactor} = 1$ since $\text{wait} = 0$

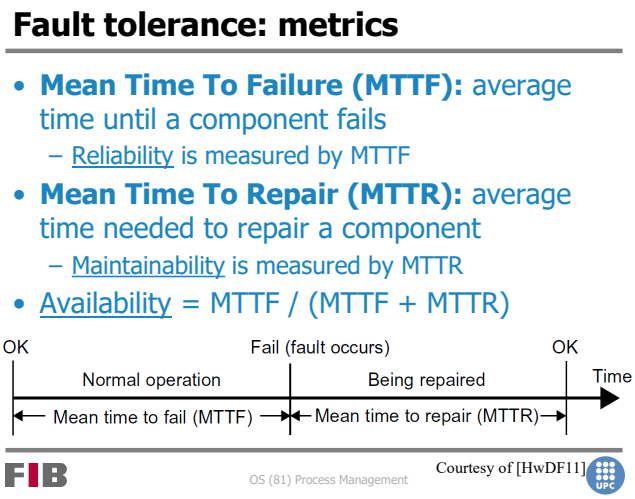
- We prioritise older jobs that arrive at $t_{y<x}$ by their arrival time, meassured by $\text{wait} = t_x - t_y$
- We prioritise short jobs measured by duration. We normalise the response time wait+duration by duration
- We respect the job arrival order by using FCFS policy to break ties (jobs arriving at the same time t have an order)

Process Resilience

Requirements

- Reliability: How long can the system operate without failure
- Availability: Percentage of time that the system is available to the user
- Maintainability: How easily can a failed system be repaired

1 Resilience Metrics



1.1 Reliability: Mean Time to Failure (MTTF)

Average time until a component fails

1.2 Maintainability: Mean Time to Repair (MTTR)

average time needed to repair a component

1.3 Availability: Percentage of Time when Available

$$A = \frac{\text{Available Time}}{\text{Total Time}} = \frac{\text{MTTF}}{\text{MTTF} + \text{MTTR}} \tag{5}$$

2 Fault Tolerance

Increase process resilience

- Increase MTTF
  - more reliable HW (↑ cost)
  - bug-free SW (infeasible)
- Decrease MTTR
  - Redundancy: replicate HW and SW components to **mask failures** to the application
  - Recovery: correct execution is **eventually** restored

3 Redundancy

3.1 Modular Redundancy

3.1.1 Double Modular Redundancy

Replicate process or modules

- in general, $\boldsymbol{f+1}$ replicas can tolerate $\boldsymbol{f}$ **crash** or **omission** failures
  - At least **one** replica can response
- Can **detect** Byzantine Failures (**CANNOT** tolerate) by comparing output of replicas

3.1.2 Triple Modular Redundancy

Output is decided through **majority**

- in general, $\boldsymbol{2f+1}$ replicas can tolerate up to f Byzantine failures
  - Apart form the f lies, we still have $f+1$ truth tellers, hence the majority is the truth
  - Needs at least $\underbrace{f}_{\text{under } f \text{ crushes}} + \underbrace{1}_{\text{available truth-teller}}$ nodes to operate normally (distinguish lies)

Example: If the mean time until each replica has a Byzantine failure is 1 week and a failure typically takes 7 hours to fix, what is the availability (in good form) of the replicated service?
Solution

- To tolerate Byzantine Failure, we need **at least TWO** replicas to operate **normally** to reach majority
- $P(\text{1-Available}) = \frac{\text{MTTR}}{\text{MTTR+MTTF}} = \frac{7\times 24}{7\times 24+7} = \frac{24}{25}$
- $P(\text{1-Fail}) = 1 - P(\text{1-Available}) = \frac{1}{25}$
- $P(\text{All-Available}) = 1 - (P(\text{2-Fails-1-Available}) + P(\text{3-Fails})) = 1 - \left(\binom{1}{3} P(\text{1-Fail})^2 P(\text{1-Available}) + P(\text{1-Fail})^3\right)$

3.2 State Machine Replication (SMR)

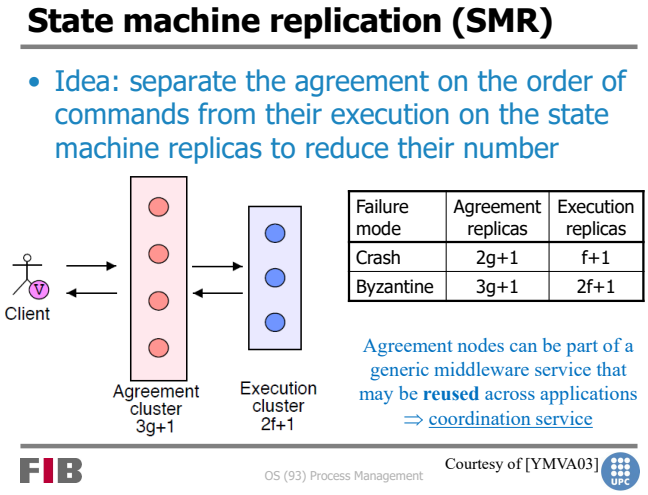
- Client requests to **ALL** the replicas
- Replicas are **deterministic**: If run the same commands, their states and outputs should be identical
- Clients wait for
  - Masking crash: 1 reply
  - Making byzantine failure: $f+1$ **identical** replies

SMR in *asynchronous* environment
Achieved within the cluster

- Assume asynchronous systems
- Can accept concurrent requests from clients
- Number of replicas **higher than** modular redundancy
- Need to decide the ordering or concurrent events
- $2f+1$ replicas to tolerate crash or omission failures
- $3f+1$ replicas to tolerate Byzantine failures

We could separate the **agreement cluster** and the **execution cluster**

- Agreement Cluster: Consensus Algorithm
- Execution Cluster: Modular Redundancy



4 Quorum Systems

A quorum can represent the entire system

- Improve the SMR availability: only contact nodes in a quorum

D-Availability: At least one **fully correct** quorum exists all the time ($N - f \geq |Q|$)

4.1 Crash Tolerance

Tolerate f Crashes

- Number of replicas: $N = 2f + 1$
- Size of Quorum: $|Q| = \lfloor \frac{N}{2} \rfloor + 1$ (Majority quorum)
- $|Q| = \lfloor \frac{2f+1}{2} \rfloor + 1 = f + 1$
- Tolerate f crashed, if we contacted $f + 1$ nodes and f crashes, we are guaranteed to have **one** result

4.2 Crypto Byzantine Tolerance

Tolerate f Byzantine Failures

- Number of Replicas: $|N| = 3f + 1$
- Size of Quorum: $|Q| = \lfloor \frac{f+N}{2} \rfloor + 1$
- Derivation: (assume two quorums with size $|Q|$)
  - **Intersection Size**: $|Q_1 \cap Q_2| = |Q_1| + |Q_2| - |Q_1 \cup Q_2| = 2|Q| - N$
  - **At least *one correct answer* in the intersection**:
    - $2|Q| - N \geq f + 1$
    - $\Rightarrow |Q| \geq \lceil \frac{N+f+1}{2} \rceil = \lfloor \frac{N+f}{2} \rfloor + 1$
  - **D-availability**
    - $N - f \geq |Q|$
    - $\Rightarrow N - f \geq \frac{N+f+1}{2} \Rightarrow N \geq 3f + 1$

4.3 Verbal Byzantine Tolerance

Tolerate f Byzantine Failures

- Number of Nodes: $|N| = 4f + 1$
- Size of Quorum: $|Q| = \lfloor \frac{2f+N}{2} \rfloor + 1$

- Derivation
  - **Intersection Size**: $2|Q| - N$
  - **At least *majority of correct answers* in the intersection**:
    - $2|Q| - N \geq 2f + 1$
    - $\Rightarrow |Q| \geq \left\lceil \frac{2f+1+N}{2} \right\rceil = \left\lfloor \frac{2f+N}{2} \right\rfloor + 1$
  - **D-availability**:
    - $N - f \geq |Q|$
    - $\Rightarrow N - f \geq \frac{N+2f+1}{2} \Rightarrow N \geq 4f + 1$

5 Recovery Strategies

Two types of recovery
- **Forward Recovery**: find a new error-free state
- **Backward Recovery (Rollback)**: Bring the system back into a previous error-free state

5.1 Backward Recovery

- Checkpoint/Snapshot: Record the system state periodically onto stable storage to establish recovery points
- Downtime: Upon failure, reboot or bring a spare host online
- Recover: Recover the system by rolling back to the previous checkpoint

States in distributed programmes
- Individual states
- Communication channels

Inter-process dependencies causes non-failing processes to rollback just to avoid **orphan messages**

5.2 Global Consistent Recovery State

Processes must identify a **recovery line**
- The most recent **consistent collection** of checkpoints
- Consistent: without orphans

Techniques
- Independent (uncoordinated) Checkpointing
- Coordinated (consistent) Checkpointing

5.2.1 Independent Checkpointing

Each process takes checkpoints independently.
- ↑ Take a checkpoint when it is most convenient
- ↓ Requires **global coordination** to compute recovery line
- ↓ Potential domino effect: processes may roll back to the beginning
- ↓ Possible useless checkpoints (never be a part of a global consistent state)
- ↓ Each processes must keep multiple checkpoints (we don't know how far we will fallback to previous checkpoints)

Upon failure
- Rollback **each process** to its most recent saved state
- Check if the local states **jointly** form a "recovery line" (**consistent global state**)

- If not consistent, keep rolling back (Question: all processes roll back by one checkpoint uniformly? Otherwise we will need to explore a combinational search space…)

5.2.2 Coordinated Checkpointing

- ↑ Saved state is automatically globally consistent
- ↑ No domino effect because every process can start at he most recent checkpoint
- ↑ Each process only maintain one checkpoint on the stable storage
- ↓ Large latency (blocking coordination)

Blocking Coordination

- Coordinator multicasts a **checkpoint-request** message
- Participants receive the **checkpoint-request** message
  - **Freeze execution**
  - **Queues** every subsequent incoming messages
  - **Ackowledge** the coordinator
- Corrdinator: after gathering all ACKs, broadcast a **checkpoint-done** message
- Participants receive the **checkpoint-done** message
  - resume all processes

Non-blocking Coordination: Chandy & Lamport's Snapshot Algorithm

5.2.3 Recovery Message Handling

- Normal Messages: sent and received **before** the recovery line
  - not effected by the rollback
- Lost Messages: **sent before** the recovery line but **received on the other side** of the line
  - must be **replayed** from the log (it is queued)
- Delayed Messages: **sent after** the recovery line but have **not yet arrived**
  - discard on arrival after rollback to avoid orphan messages
  - it will be sent after recover in re-execution
- Vanished Messages: sent and received **after** the recovery line
  - no action needed, will be sent after recover in re-execution

5.3 Failure Prediction

- "Precursor" events can provide a good sense of when a PE might fail in the future
- Checkpoint proactively only when a failure seems likely in the short term
- Migrate all processes running in a PE when a failure seems likely in the short term

5.4 Log-based recovery

- Combine checkpoints with message logging, similar to "lost messages"
- Processes take infrequent checkpoints
- Processes log **all the received messages** to to local stable storage (pessimistic)
- Upon a failure
  - the crashed process rolls back to its latest checkpoint
  - **reply** the messages received **since that checkpoint** in the right order
- ↑ If a process fails, other process does not need to roll back to ensure consistency

Virtualisation

- Provides illusion of multiple dedicated systems

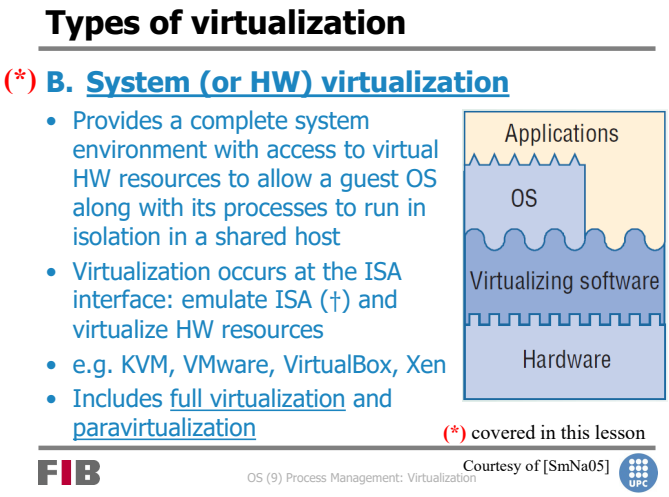
- Isolates programmes from the system and other programmes
- Virtual environments can be created and destroyed with little overhead
- Allow agile and fine-grain dynamic resource provisioning
- Enables fault tolerance for programmes

1 Process Virtualisation

2 System/Hardware Virtualisation

Provides **complete** system environment to allow a **guest OS** to **run in isolation** in a shared host.
e.g. KVM, VMware
Virtualisation at ISA interface: Emulate **ISA** and virtualise **HW resources**

- ↓ Overhead to run VM instances due to emulation
- ↓ Slow VM initialisation due to VM creation and loading of guest OS
- ↓ Great size of VM images and probable replicated contents (same OS)



2.1 Full Virtualisation

Use **hypervisor** to allow **unmodified** guest OS to run in isolation
Requirements

- **Safety**: Only VVM should manage the HW (guest is limited to its virtual space)
  - Sensitive instructions must be emulated to make OS work correctly
- **Fidelity**: Programmes on VM execute identically to their execution on HW (except for timing)
  - Emulation is transparent to the guest OS
- **Performance**: non-sensitive instructions can be executed directly (if ISA is the same)

2.1.1 Bare-metal Hypervisor (Type I)

- The hypervisor is the **only** programme running on the **most privileged mode**
- Provide virtualisation and OS functionality
  - CPU scheduling
  - Memory and I/O management
  - device drivers
- Examples: VMware ESX, Xen, Hyper-V
- Typically deployed on server systems and production environments

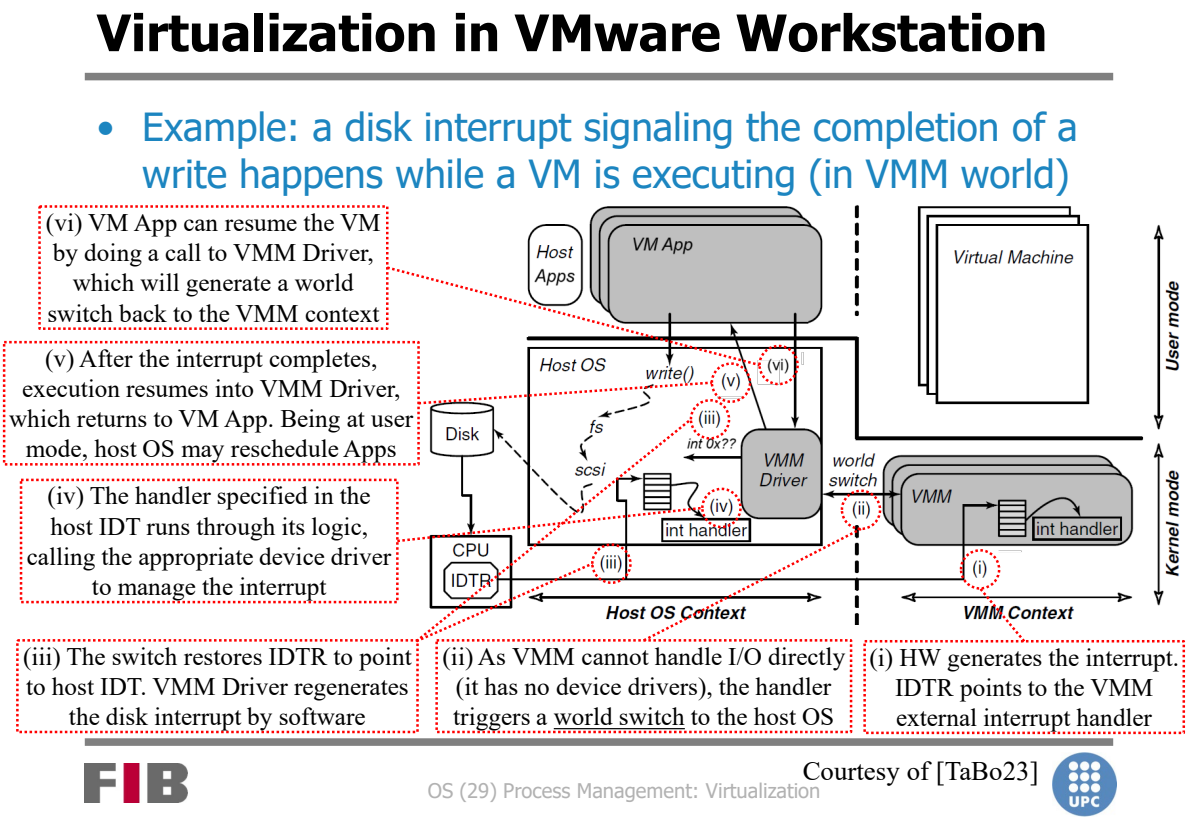
2.1.2 Hosted Hypervisor (Type II)

- Run as a **user-level process** on top of an existing OS
- Depends on the host OS for much of its functionality
- Examples: VMware Workstation, VirtualBox
- Typically deployed on commodity devices and development/testing environemtns

2.1.3 Modern Hosted VMMs

- **VMM in Ring 0**
  - A sort-of Type I Hypervisor
- **Host OS in Ring 0**
  - **VMM App** that runs in user mode of the host OS
  - **VM Driver** that runs as a **kernel module** in the host OS
    - Install trap handlers in the kernel and se trap-and-emulate
    - Does the **world switch** from the **host world** to the **VMM world**

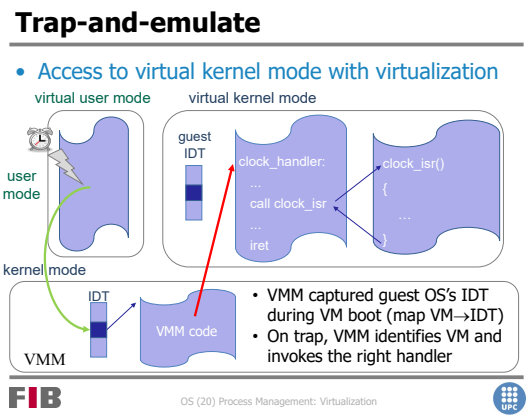
The host OS is responsible for scheduling resources between **VMs** and native applications.
The host OS is responsible for handling I/Os (VMM has no device driver, it will switch back to the host OS)

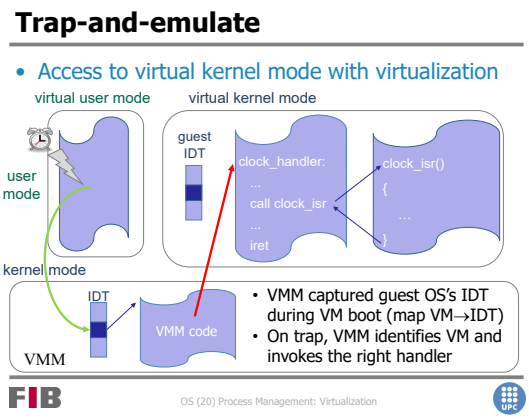


2.1.4 Sensitive Instructions: Trap-and-Emulate

- **Requirement**: SensitiveInstructions $\subset$ PrivilegedInstructions
  - In x86, some sensitive instruction do not trap when run in user mode (e.g. `popf`)
  - Solution
    - Binary Translation + Protection Rings (At first)

  - Hardware-assisted Virtualisation (Later on)
- Privileged instructions issued by the guest OS will generate a trap in user mode
- VMM intercepts the traps and emulate the trapping instruction against the virtual machine state machine
- Execution mode
  - Kernel Mode: Hypervisor
  - User Mode: 1. Virtual User Mode, 2. Virtual Kernel Mode





2.1.5 Binary Translation

Code in **virtual user mode**: directly executed
Code in **virtual kernel mode**: emulated (interpreted)

- **Scanned** on the fly for basic blocks
- Blocks are rewritten to **replace** sensitive instructions with new code that **emulates their effect** against VM state
- Blocks can be **cached**

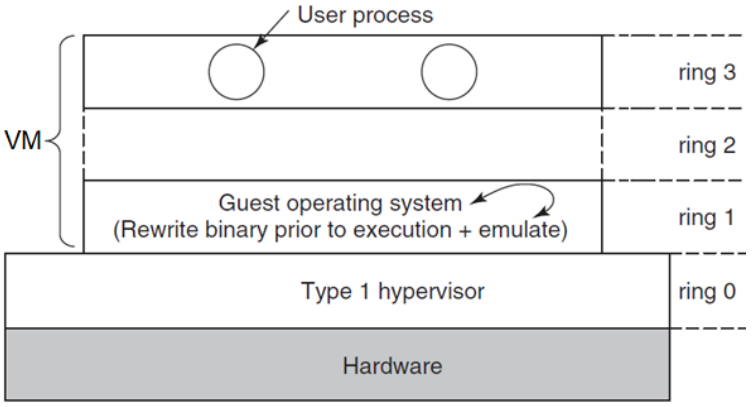
2.1.6 Protection Rings

On x86

- Guest User Process: Ring 3
- Guest OS: Ring 1
- Hypervisor: Ring 0

Protection rings

- As x86 supports 4 protection modes (rings), the guest OS can be placed in ring 1



As ring 3 > ring 1, any attempt to access the guest OS memory from a user process will lead to an access violation (*)

As ring 1 > ring 0, privileged instructions within the guest OS will still trap to the VMM

(*) Note that if the system has only two modes, the guest OS runs in user mode and must use memory protection to prevent the access of OS data structures from user processes

2.1.7 Hardware-Assisted Virtualisation

2.1.7.1 VMX mode

- Virtual Machine Monitor (VMM): **VMX Root**
- Guest OS: **VMX Non-Root** but remains in (Non-root) Ring 0

2.1.7.1.1 VMX Transitions

- **VM Entry**
  - VMM (VMX Root) -> Guest (VMX Non-Root)
  - `vmlaunch`: initial entry
  - `vmresume`: subsequent entry
- **VM Exit**
  - Gurst (VMX Non-Root) -> VMM (VMX Root)
  - Entered on **Trap**

2.1.7.1.2 VM Control Structures (VMCS)

Stored **by hardware**

1. **Guest** and **host** states
   - **Guest-State Area**: save on *vm exit*, load on *vm entry*
   - **Host-State Area**: load host state on *vm exit*
2. Behaviours of instructions in VMX mode
   - All **sensitive instructions** can be set to **trap** when executed on **Non-Root** mode
   - We can trap-and-emulate on x86 now

2.2 Paravirtualisation

Modify source code of the guest OS to replace **sensitive instructions** with **hyper calls** to the VMM

- ↑: Lower overhead than full virtualisation
- ↓: Guest OS is tied to VMM (use VMM-agnostic hypercalls)

2.3 Virtualisation of OS functions

2.3.1 VCPU Scheduling (Double Scheduling)

- Guest OS: Schedules processes onto vCPUs (each guest is assigned some vCPUs)
- VMM: Schedules vCPUs onto the physical CPUs

Typically based on proportional-share scheduling: better to ensure that **all** VMs can make progress (fair CPU scheduling, no starvation)

2.3.1.1 Borrowed Virtual Time (BVT)

- Allocate CPU time in proportion to the **weight (CPU Share)** assigned to a vCPU

2.3.1.1.1 Virtual Time

Gives all runnable vCPUs equal amount of virtual time ⇒ always run vCPUs with **lowest virtual time**

- Consumed inversely proportional to weight

For each vCPU_i, track its virtual time (A_i)

$$A_i \mathrel{+}= \text{RealRunningTime} \times \underbrace{\left(\frac{1}{w_i}\right)}_{m_i} \tag{6}$$

- RealRunningTime meassured by timer interrupt **minimum charging unit**
- w_i: weight, CPU consumption share
- $m_i = \frac{1}{w_i}$ and normalised to be an integer (w_i is the wright of vCPU)
- ⇒ vCPU with higher weight "consumes virtual time" slower

2.3.1.1.2 Context Switch Condition

switch from i to j if

$$A_j \leq A_i - C \times m_i \tag{7}$$

- C: Context switch allowance (permission to run "overtime" beyond another runnabCle vCPU)
- Prevent ping-pong switches on every timer interrupt

2.3.1.1.3 Starvation Avoidance: Scheduler Virtual Time (SVT)

Avoid Starvation after waking up from **Voluntary** Sleep
must be from a VOLUNTARY **sleep**

$$\text{SVT} = \min\{A_j : j \in \text{Active(VCPUs)}\} \tag{8}$$

After waking up, set $A_i = \max(A_i, \text{SVT})$ to avoid very low A_i

2.3.1.1.4 Time-Critical Support: Effective Virtual Time (EVT)

Time-critical Task support, if enabled, allow it to run more by decreasing the existing virtual time

- **Warp Values**: $\text{warp}_i \in \{0, 1\}$ flag to enable/disable warp (on demand)

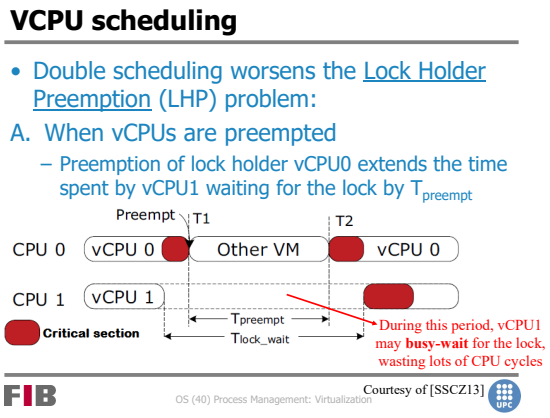
- **Borrow VT from Future Allocation** (W_i): Specify how much time needed from the future
- **Effective Virtual Time**

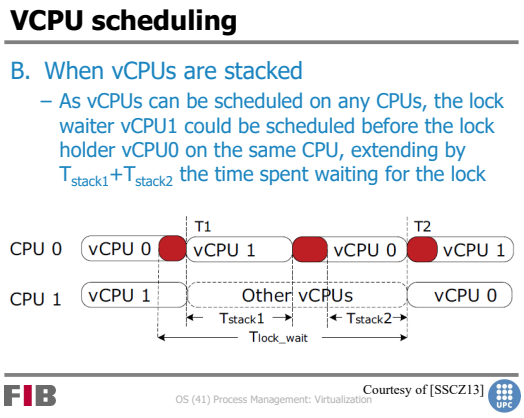
$$A_i = \begin{cases} A_i - W_i & \text{if warp}_i \\ A_i & \text{if } \neg\text{warp}_i \end{cases} \tag{9}$$

2.3.1.2 Lock Holder Preemption (LHP) problem

vCPU preempted while holding the lock

- Preempted: vCPU_1 on pCPU_a holds the lock and is preempted, vCPU_2 on pCPU_b busy wait during this period
- Stacked: vCPU_1 on pCPU_a hols the lock and is preempted, vCPU_2 scheduled on the same cCPU_b and then busy wait





2.3.1.3 Pause-Loop Exiting (PLE)

Hardware support to detect busy-waiting vCPUs

- Monitor the number of `pause` instructions
- VMM gets trapped so it can preempt busy-waiting vCPUs and yield to another one
- ↓ PLE events are not enough to determine **who** to yield to

2.3.1.4 Paravirtualised Spinlocks

Modify spinlock implementation in the **guest OS** to add **hypercall**s to the VMM

- Inform the VMM that a lock is acquired → **prevents** the **preemption** of this vCPU
- **Yield** to another vCPU when lock **cannot** acquire quickly
- ↑ VMM can track the current lock holder (thanks to hypercall) and can yield to the most appropriate vCPU
- ↓ Modification of guest OS

2.3.1.5 Gang Schedulling

Scheduling vCPUs of the **same VM** simultaneously (if #vCPUs < #CPUs)

- ↑ Avoid stacking and busy-waiting
- ↓ Fragmentation, priority inversion

2.3.2 Memory Virtualisation

2.3.2.1 Software Memory Virtualisation

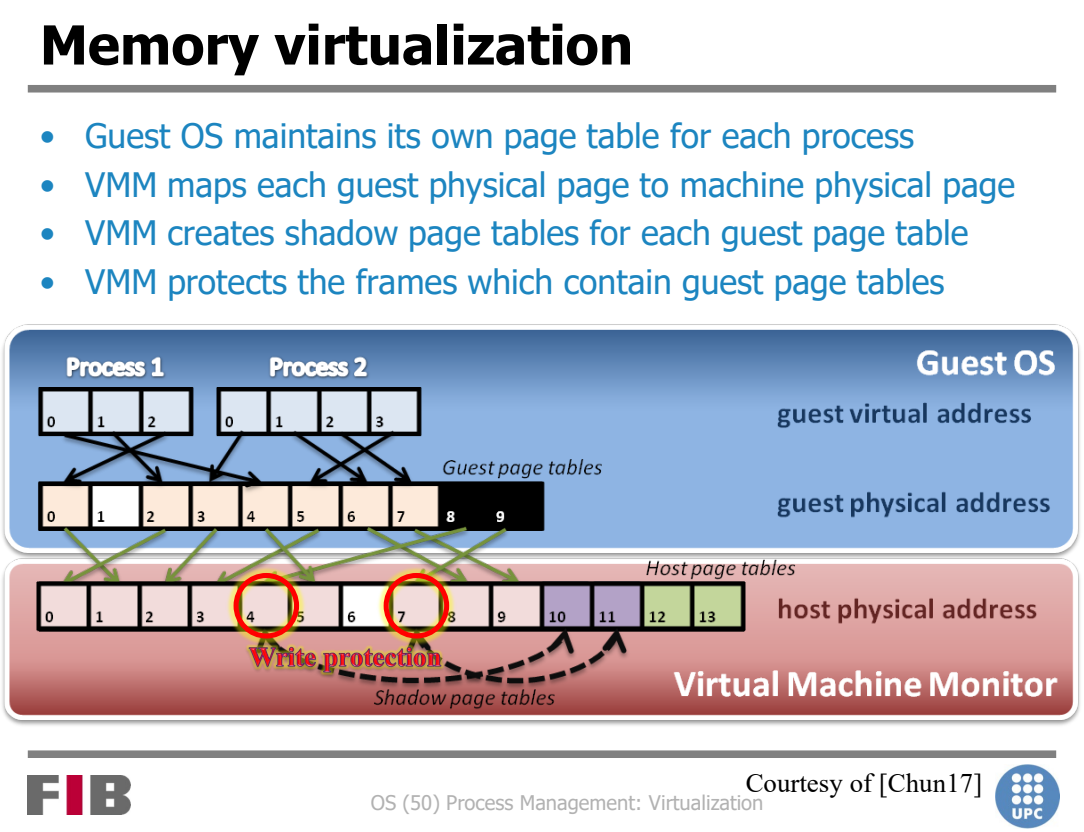
VMM maintains

- **Shadow Page Table**: **per-process** maps from (virtual) **Guest Physical Addresses** directly to (real) **Host Machine Addresses**

- **Virtual Page Table Base Register (vPTBR)**: points to the **shadow** page tables of the current active guest OS

The Hardware actually uses the **Shadow Page Table** for the address translation.
VMM must **Keep shadow page table in sync**: Capture guest's attempt to install or update guest page tables

- ↓ Manipulating shadow page tables is **expensive**
- ↓ Space overhead due to shadow copying



Full virtualisation

- Instructions to modify **virtual** PTBR are privileged, hence traps into VMM
  - e.g. `MOV CR3, RAX` is privileged. Invocation by guest OS will trap into VMM
- Guest page tables are write-protected, hence modification by the guest OS (in user mode) traps into the VMM

Paravirtualisation

- Use hypercalls to inform VMM about changes to page tables

2.3.2.2 Hardware-Assisted Memory Virtualisation

EPT (intel) / NPT (AMD)
MMU offers two **nested levels** of address translation

- **Guest page tables** (maintained by the guest OS)
- **Extended page tables** (maintained by the VMM)

MMU can walk directly through guest and extended page tables

- ↑ Better performance by avoiding shadow paging
  - No VMM involvement to sync tables

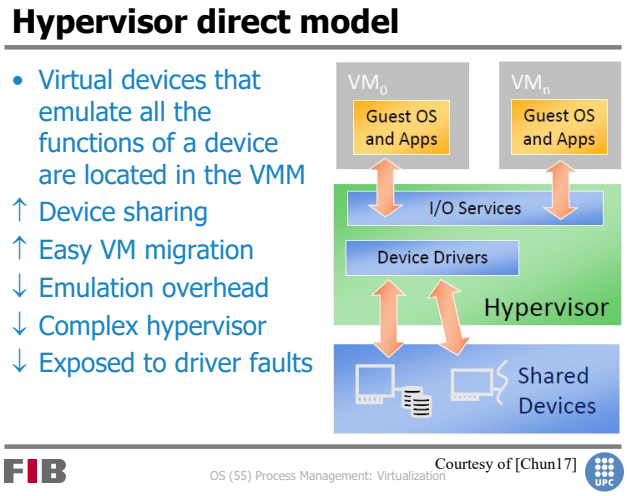
  - Less space needed: one EPT **per guest OS** instead of one shadow page table **per process**
- ↓ Longer address translation time, TLB misses are more costly

2.3.3 I/O Virtualisation

- Decouples virtual from physical I/O activity through software layer
- Guest OS is offered with virtual I/O devices
- VMM intercepts guest OS accesses to these virtual devices
- VMM emulates virtual I/O devices (using physical HW is needed)

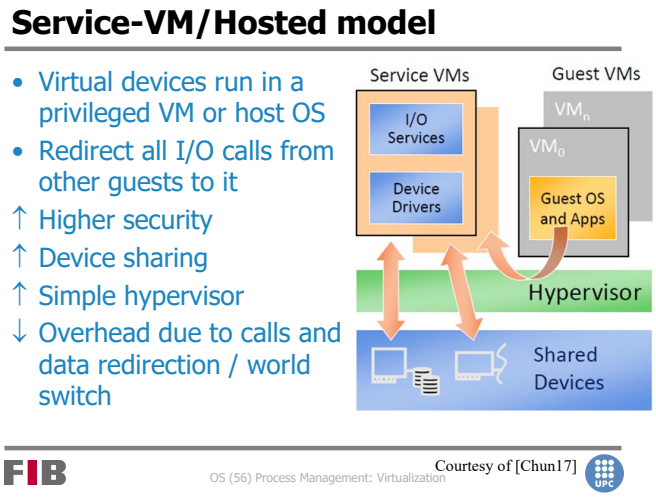
2.3.3.1 Hypervisor Direct Mode

- VMM contains **virtual devices** that **emulates all the function** of the device
- ↑ Device sharing
- ↑ Easy VM migration
- ↓ Emulation overhead
- ↓ Complex hypervisor
- ↓ Exposed to driver faults: effects all VMs



2.3.3.2 Service-VM/Hosted model

- Virtual devices only run on **privileged (service) VMs**
- Guest VMs' I/O calls are **redirected** to the service VMs by the hypervisor
- ↑ Higher security
- ↑ Device sharing
- ↑ Simple hypervisor
- ↓ Overhead due to **calls**, **redirection** and **world switch**



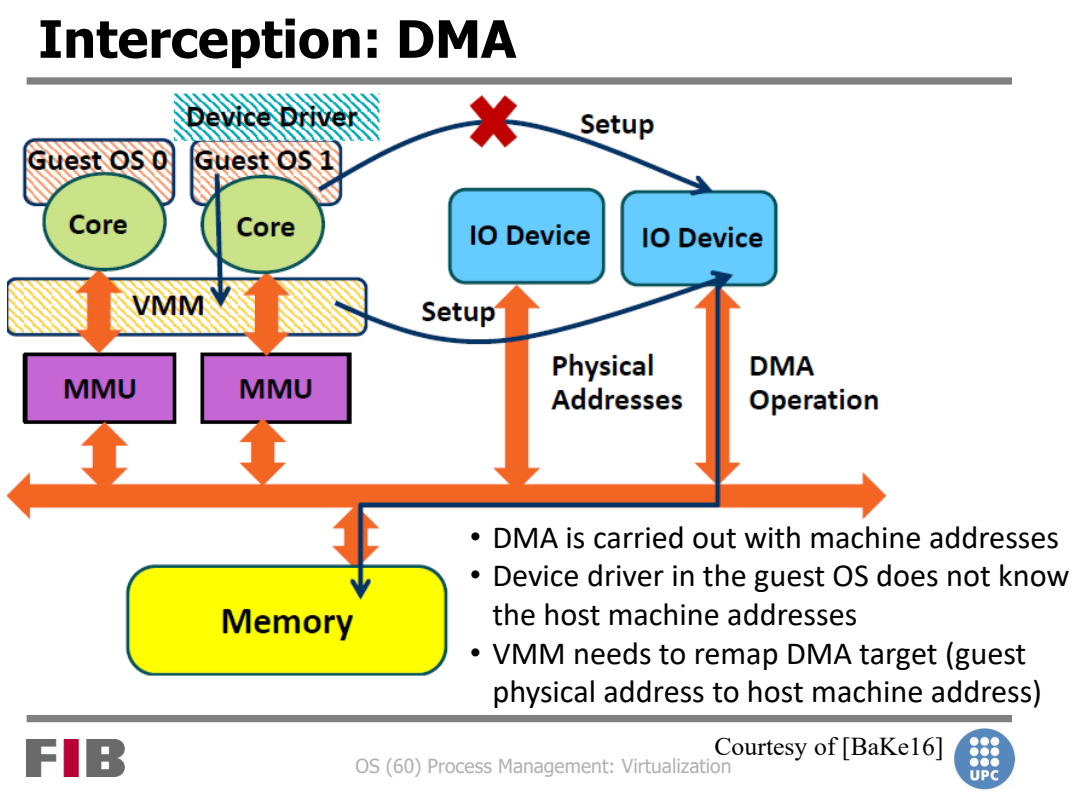
2.3.3.3 Pass-Through Model

- Physical device is **directly** and **exclusively** assigned to a particular guest OS (other VNs cannot use the same device)
- ↑ Removes emulation overhead
- ↑ Simpler hypervisor
- ↓ **Strong coupling** between guest OS and HW
  - Harder VM migration
  - Harder device sharing

2.3.3.4 I/O Interception in Full Virtualisation

Trap-and-emulate I/O operations of the guest OS

- Memory-Mapped I/O
  - Devices mapped in **regular memory**
  - access with `load`/`store` instructions
  - Use memory page protection to generate trap
- Port-Mapped I/O
  - Mapped in a **separate address space**
  - access with `in`/`out` instructions
  - these instructions are privileged, and generates trap
- Direct Memory Access
  - these devices bypasses CPU, needs **host machine addresses**
  - needs **address remapping** for the target



2.3.3.5 I/O Interception in Paravirtualisation

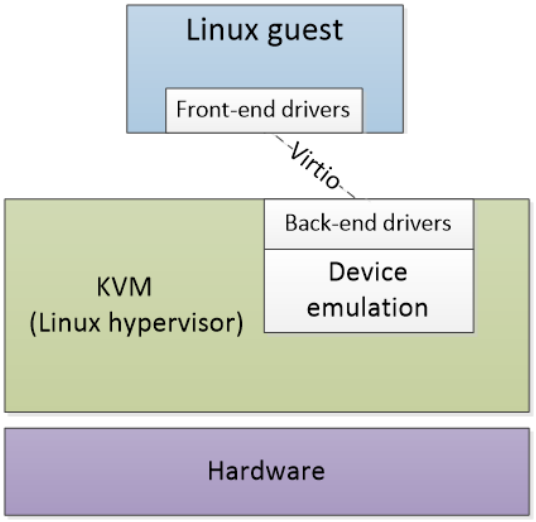
Redesign virtual devices to optimise them for the virtualisation layer

Split the virtual device (driver interface to the guest) **from the device driver** (interface with the actual device)

- Make hypercalls to VMM to initiate requests
- Buffers are shared between the guest OS and the hypervisor
- ↑ Lower overhead
- ↓ Specific drivers for each guest OS
- e.g. virt-io for KVM/QEMU (paravirtualisation I/O framework)

Interception: Paravirtualization

- Redesign virtual devices to optimize them for the virtualization layer esp. regarding interception
  - A. Make hypercalls to VMM to initiate requests
  - B. Buffers shared between guest OS and VMM
- ↑ Lower overhead
- ↓ Specific drivers for each guest OS (less portable)



e.g., **virt-io** (paravirtualized I/O framework for KVM/QEMU)

FIB OS (61) Process Management: Virtualization Courtesy of [Jone10] UPC

2.3.3.6 Hardware-Assisted I/O Virtualisation

VT-d (Intel)

- I/O MMU hardware virtualises memory
- Provides **address translation** for DMA (Guest PA to Host Machine Address)
- Provides **memory protection** for DMA
- Provides **dynamic remapping** of interrupts

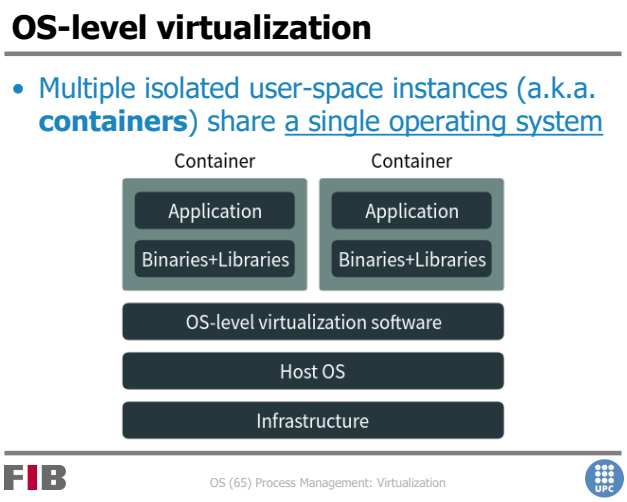
2.4 OS-Level Virtualisation (Containers)

Seregate and isolate multiple user-space instances (containers) within a **single OS**

- ↑ Little overhead
- ↑ Minimal startup/shutdown
- ↑ Low resource requirement, high scalability
- ↓ All containers (in the same machine) **must** use the **same os**
- ↓ Containers **cannot** have their **own modules**

Each container feels list a virtual machine, but **shared the same OS**

- own process space (PID, etc)
- own network interface
- run stuff as root
- install packages, shells etc



Linux Namespaces

- types of namespaces `pid`, `net`, `mnt`, `uts`, `ipc` ,`user`, `time`, `cgroup`
- Namespaces are created with `create`, `unshare` system calls, with flags `CLONE_NEW*`
- `setns` call allows joining existing namespace
- Namespace is **deleted** when last process leaves
- `pid`: process have their own
  - **process numbering** and
  - can only **kill** other processes **in the same namespace**
  - `ps` sees all processes with external PIDs unless we mount new `/proc`
  - `kill` uses internal PIDs
- `net` process get their own private network stack, including
  - network interfaces
  - routing tables
  - ip-table rules
  - sockets
- `uts`: processes can have their own host names
- `mnt`: processes can have their own
  - root file system and
  - private mounts (`/proc`, `/tmp` etc)
- `ipc`: processes can have their own
  - message queues
  - semaphores
  - shared memory
- `user`: maps process UID-GID
  - e.g. **privileged** user ID **inside** and **unprivileged** user ID **outside**
- `time`: processes can see different clock
- `cgroup`: processes have their own set of `cgroup` root directories
  - Resource Limiting
    - #CPUs, memory, #PIDs, block I/O speed, network packets
  - Resource Prioritisation: when under heavy load, which cgroup to give priority (e.g. 80% to database group and 20% to backup group)
  - Resource Usage Monitoring
  - Process Control (e.g. suspend, restart all processes with in a cgroup at once)

Capability: divide privilege into distinct units to be independently enabled/disabled
- A process with root privilege can be limited to get only the minimal permissions it needs to operate

Seccomp: define which syscalls are allowed to be executed by a process

2.5 Application-Level Virtualisation (Programming-Environment Virtualisation)

Abstract away details of the underlying HW and OS, allow programme to execute the same way on any platform. e.g. JVM

3 Virtual Clusters and Resource Management

- Built with VMs from one or more physical clusters
- VMs in a virtual cluster are **interconnected logically** by a **virtual network**
- Virtual clusters may grow and shrink dynamically

3.1 VM Deployment

1. Prepare disk image
   - Template images
   - Use COW to avoid copying the whole image
2. Configure VM
   - UUID, VM name, disk image, network settings, allocated CPUs and memory
3. Choose the destination host **(VM Allocation)**
   - Fulfill the VM requirements
   - Balance load, consolidate VMs, reduce energy consumption

3.2 Live VM Migration

Requirements
- Negligible downtime
- Lowest network bandwidth consumption
- Do not disrupt other services in host
- Must migrate
  - Memory
    - Could be compressed to reduce network bandwitdh
    - Pre-Copy and Post-Copy
  - Storage
  - Network

3.2.1 Memory Migration

3.2.1.1 Pre-Copy

- ↑ No need to fetch pages from the source list
- ↑ An up-to-date state of the VM is retained at the source host during migration
- ↑ If destination fails during migration, VM **can be recovered**
- ↓ Migration may **never end** if source $V_{\text{dirty}} > V_{\text{transfer}}$
- ↓ Can transfer the **same page multiple times**, if the page is **dirtied repeatedly** at host during migration

Steps (Serial Steps)

1. **Pre-Migration and Reservation**
   - VM Allocation
     - Determine migrating VM
     - Determine destination host
   - initialise a VM on he target host
2. **Memory Push (iterative Pre-Copy)**
   - In this stage, the **source** machine is still processing requests, but at a slower speed
   - Transfer **all** memory pages in the first round
   - Copy **dirty** memory pages (data which is changed) in successive rounds
   - Repeat until $\#\text{DirtyPages} < \text{Threshold}$
     - $V_{\text{transfer}} > V_{\text{dirty}}$, assume not memory intensive
     - **CANNOT** guarantee termination when $V_{\text{transfer}} \leq V_{\text{dirty}}$
3. **Stop and Copy**
   - **Suspend** VM on the source host (T_{off})
     - execution of this step is **serial** (in exams)
   - Network connection is **redirected** to the new VM
   - Transfer the **remaining** dirty pages
     - Transfer VCPU and Network States as well
4. **Commitment and Activation**
   - VM reloads state and resumes its execution on the destination host (T_{on})
   - Remove orginal VM from the sources host

3.2.1.2 Post-Copy

- ↑ Live migration ends in a finite time
- ↑ Sends each page **exactly once** over the network
- ↓ VM **state is distributed** over both source and destination hosts ⇒ If destination fails during migration, the **VM cannot be recovered**
- ↓ Latency of remote page fault (fetching pages from the source host)

Steps

1. **Pre-Migration and Reservation** (SAME)
2. **Stop and Copy (Down)**
   - Suspend VM and transfer **minimal execution state**
   - $T_{\text{OFF}} + T_{\text{context}} + T_{\text{ON}}$
3. **Activation**: resume VM at destination host
4. **Running VM and Migrate Memory (Push)**
   - Demand Paging (Remote Page Fault)
     - If VM tries to access a memory page that has not been migrated, generate a page fault
     - Fetch the missing page with high priority
   - Proactive Page Push
     - The source continues to push remaining memory pages to the destination in the background until all pages are transferred
   - Notes
     - Here, every memory page is copied exact once, so the total time for memory transfer is $T_{\text{migrate}} = \frac{\text{MemorySize}}{\text{BandWidth}}$
     - $T_{\text{Migrate}} = T_{\text{PageFaultInstructions}} + T_{\text{RegularInstructions}}$
     - If we know the useful instructions executed during this migration, we can calculate the rest of the instructions to execute on the new host

  - $\mathrm{NI}_{\mathrm{migr}} = \mathrm{NI}_{\mathrm{PageFault}} + \overbrace{\overbrace{\mathrm{GIPS}(h_2)}^{\text{Dest Full Speed}} \cdot \widehat{R}}^{\%\text{ used for normal execution}} \cdot T_{\mathrm{RegularInstructions}}$
  - $\mathrm{NI} = \mathrm{NI}(h_1) + \mathrm{NI}_{\mathrm{migr}} + \mathrm{NI}(h_2)$
5. **Termination**
   - When all pages are transferred, the original VM is removed

3.2.2 Storage Migration

- **Shared Storage Model**
  - Virtual disks resides on a single shared volume accessible by all hosts
  - ↑ No copy needed
  - ↓ Required HW or SW support for shared storage
- **Shared-Nothing Mode**
  - Virtual disks are moved across different storage elements
  - ↑ Can worl with any kind of storage support
  - ↓ Must be optimised to avoid huge downtimes due to the current trend on disk capacity

3.2.2.1 Shared-Nothing: Snapshotting

1. Take a snapshot as **Base Disk** (source)
   - The base disk becomes read-only
   - The VM continues to work in a delta disk
2. Redirect write to **Delta Disk** or **Redo Log** (source)
3. After the copy **Base Disk**, **consolidate the Delta Disk into Base Disk**; while VM is working in **another** delta disk (source)
4. Repeat until the amount of data in the delta disk becomes smaller than the threshold
5. Suspend VM and consolidate the final delta disk

3.2.2.2 Shared-Nothing: Dirty Block Tracking

Same protocol as **memory pre-copy** but copies disk blocks instead of memory pages

- Instead of redirecting writes to a new delta disk, we writes to the **original disk**
  - We don't need to keep a strictly snapshot of the state before migration. While we are transferring data, we allow writes to go through (The old data will be overwritten anyway, no need in preserving them)
- It is tracked only by bitmap, less storage overhead

3.2.2.3 Shared-Nothing: I/O Mirroring

Mirroring writes to the source and destination concurrently

- Writes to region already copied: mirror to source and destination
- Writes to region not copied: to source
- Writes to region currently being copied: Delay until copy finishes and then mirror to both source and destination

3.2.3 Network Migration

VM keeps its virtual MAC and IP addresses

- Migrating within the **same local subnet**
  - Source host broadcasts ARP reply to advertise (to the **local network**) that the IP has moved to a new location
  - Same in-flight packets may get lost, but TPC retransmissions will take care of that
- Migrating across subnets, an **indirect layer** must exist
  - e.g. Mobile IP: a node on the source network forwards packets to a care-of address on the destination network

3.3 VM Allocation

- Decide mapping of VMs to physical hosts
- Migrations allow moving VMs among hosts
- Can be viewed as a **fully dynamic packing problem** (NP-hard)
- Simple **bin-packing heuristics** have been adapted
  - e.g. First Fit and Best Fit heuristics
  - VM allocation is harder than bin-packing
- Comparison between VM Allocation and Bin-Packing
  - Several resource dimensions
    - CPU, memory, disk
  - Performance degrades when the physical host is (close to) overloaded
  - Physical hosts have different capabilities
  - #hosts is finite, constraining the #VMs that can be allocated
  - #VMs and #hosts are volatile (changes over time)
  - VM migrations incur a performance cost

3.4 VM Consolidation

- Consolidate VMs running on underutilised hosts (Enhances utilisation)
- Live VM migration supports dynamic consolidation
- Suspend idle hosts to lower operational costs (e.g. electricity)

3.5 VM Load-Balancing

- Balance the VMs across hosts according to their changing resource requirements
- Spread the VMs to foster parallelism and dependability
- Live VM migration supports dynamic load balancing

3.6 VM Elasticity

- Dynamically and transparently scale up/down VMs that provide a service
- Horizontal: increase number of VMs among hosts
- Vertical: increase the size of the VM

3.7 VM Checkpoint/Recovery

- Upon a host failure, VMs are automatically resumed from their last checkpoint on **other physical hosts**
- Live VM migration allows **proactively** moving VMs when a failure seems likely in the short term

3.8 VM Replica Failover

- Maintain several replicas of the VMs on different hosts
- Upon a VM or host failure, service is automatically resumed from one of the replicas with **no downtime**

In-Memory Data Management

1 Desktop

Cache Coherence Protocols

- MSI: Modified-Shared-Invalid
- MESI: Exclusive-MSI
  - Exclusive: Read cache line is owned by 1 cache
- MOESI: Owned-MES
  - Owned: Modified cache line may be shared by others
- MOESIF: Forawrd-MOESI
  - Forward: Shared cache line may be forwarded to others (avoid broadcast the bus)

2 Pre-WSC

N nodes, multiple processors
- N = 2-100 nodes
- Each node has its **own** memory chip **(Multiple address spaces)**
- Local-area network

2.1 Distributed Address Space

- @Spaces of other nodes are not accessible
- Message passing mechanism needed (e.g. MPI)
- Explicit communication required for the programmer

2.2 Shared Address Space

- SW/HW layer
  - SW: Distributed Shared Memory (DSM)
  - HW: Non-Uniform Memory Access (NUMA)

2.2.1 Replicated Address Space

- Needs consistency protocol
- Memory access detection
  - Processor
    - Specific instructions: Load/Store
      - Generates bus operations
    - Loaded into cache (keep state)
  - Middleware
    - virtual memory
      - page 4MB
      - Page tables (memory protection, page fault)
- Need a **directory** to keep page stage in global @Space
- Each node has local caches (two levels of replica)

Strong consistency: All replicas updated before a write completes Problems
- Emulate memory bus
- Latency of writes
- Local caches
- Cache line size
- False sharing

Example: Treadmarks
- Software DSM
- Allow wries on different memory regions

- Allow writes on **same** memory region
  - Calculates diffs and merge changes
- Programmer needs to **mark all critical regions**
- Based on locks, groups a set of addresses and operations
- Lasy Release Consistency Protocol

2.2.2 Aggregation of Address Space

- Larger @Space
- Each node is responsible for a part of the @Space
- No replicas
  - no consistency problems
  - increased latency when accessing from other nodes

3 WSC

Need replication to
- Improve **availability**
- Enhance **reliability**
- Improve performance and scalability

But, according to CAP, we have **inconsistencies**. Relax existing consistency constraints.

3.1 CAP Theorem

In a distributed system, only two of these three constraints can be satisfied
- Consistency: **Every read** receives the **most recent write or an error**
- Availability: **Every request** (by a non-failing node) must receive a **non-error** response, **without** the guarantee that it contains the most recent write
- Partition Tolerance: The system continues to operate despite an **arbitrary number** of messages being **dropped** or **delayed** by the **network between nodes**

In WSC, we choose availability and partition tolerance over consistency → Eventual Consistency.
Exam Question (21, i) In a distributed file system that implements an Immutable files semantics and enforces the availability of the whole file system, which of the following is false?
1. (✓) client will read always the last written content.
   - We enforced **Availability**, and we can (without further guarantee), enforce **Consistency**
2. (✓) File content can be updated.
   - Immutable file system: Files cannot be modified, it can be **updated** by creating a new **version** of the file
3. (✓) Any file will be accessible at all times.
   - We enforced **Availability**
4. (✗) Data will be consistent even in the case of network failures.
   - We already enforced **Availability**, now we want data **Consistency** during network **Partition**, it is impossible

3.2 Not-Only-CAP: PACELC

On internet **Partition**: Choose between **Availability** or **Consistency** Else: choose between **Latency** and **Consistency**

3.3 Eventual Consistency

- Fast writes (Local only, propagated in the background)
- Fast reads (but may be corrupted)
- Inconsistency Window
  - The period between the **update** and the moment when it is guaranteed that **any observer** will **always** see the updated value
  - Depends on the implementation, latency, network partitions, etc

3.4 Session Guarantee

A **client-centric property** that ensures **a single user/session** perceives data in a consistent way (even if the database is eventually consistent). The following satisfy this guarantee.

- Read-Your-Writes: See your own updates
- Monotonic Reads: Never see data go back in time
- Monotonic Writes: Writes are processed in the order you sent them
- Writes Follow Reads: If write after reading a value, the write is guaranteed to happen after the version you read

3.5 Data-Centric: Consistent Prefix

Readers observe an **ordered sequence** of writes, starting with the first write to the data store, for example

- current (x, y) = (0, 0)
- updates in order: write(y, 1), write(x, 1), write(y, 2)
- we can read $(x, y) \in \{(0, 0), (0, 1), (1, 1), (1, 2)\}$
- The values we read are **always** valid values (it happened in real life, as opposed to *monotonic read*)

3.6 Data-Centric: Bounded Staleness

Read operation will return values no older than **more than T minutes ago** or K versions ago. i.e. any values **written T minutes ago** must be visible to **all readers**.

3.7 User-Centric: Monotonic Read

- If a reader reads a **specific value**, all subsequent reads from the same reader will return **at least as recent as** this one
- Reader will **never** see a data rollback
- Note
  - Monotonic read is **independent**, if we write to two different objects, we will see monotonic writes on **each** of the objects independently
  - However, the collective read is **not in write order** (as opposed to *Consistent Prefix*), we could observe inconsistent data. For example
    - current: (x, y) = (2, 3)
    - updates in order: write(x, 4), write(x, 5), write(y, 6)
    - we can read $(x, y) \in \{x \in \{2, 4, 5\} \land y \in \{3, 6\}\}$
    - so there would be results like (x, y) = (2, 6), which **does not happen**
    - Results like $(x, y) = (2, 6)$ happen when we **DON'T** read x before reading y, so x is not guaranteed to be recent

3.8 User-Centric: Read-Your-Writes

The effect of write operation by the process on data item will **always** be visible to the subsequent reads by that same process
- Even if the same process connects to different replicas, this still holds

4 Consistency Protocols

(Originally,) Most common schemes are those that support **sequential consistency**
Eventual consistency schemes are common now

4.1 Process Groups

4.1.1 Hierarchical Process Groups (Primary-Based Protocols)

- Clients send request to the coordinator (Primary)
  - Clients may be able to send read requests to workers (Replicas)
- Coordinator provides the desired service
- Coordinator update the workers using **Total FIFO-Order Atomic Multicast**

Failure (F failures in total)
If the primary fails, one of the workers promoted as primary
- Crash or Omission failures, need $F + 1$ nodes

4.1.2 Flat Process Group (Replicated Protocols)

- Clients send their requests to **all replicas** using **Total FIFO-Ordered Atomic Multicast**
- Replicas process the request independently and reply (processes are **identical**)

Processes are assumed to be **identical**.
If processes are **not identical**, non-faulty group members must **reach agreement** (consensus problem)

Failure F failures in total
If replicas fail, the remaining ones continue to respond
- Crash or Omission failures: need $F + 1$ nodes
- Byzantine Failures: need $2F + 1$ nodes

4.2 Primary-Based Protocol

4.2.1 Primary-Backup Remote-Write Protocol

- All writes are done at a **fixed single-replica**
- Reads can be carried out locally

Steps
1. Client request write to **some** backup (blocking until `ack`)
2. Write is **redirected** to the primary
3. Primary ask replicas to update and wait for replicas to finish
4. Replicas update and acknowledge the primary
5. (Primary or Replica?) Reply the client to mark complete
   - If it is replica, how does the replica know that the update is complete across all replicas? (Primary sent message? Query all replicas?)
   - The replica can replay to client and primary? Then we will not have a write majority.

Analysis

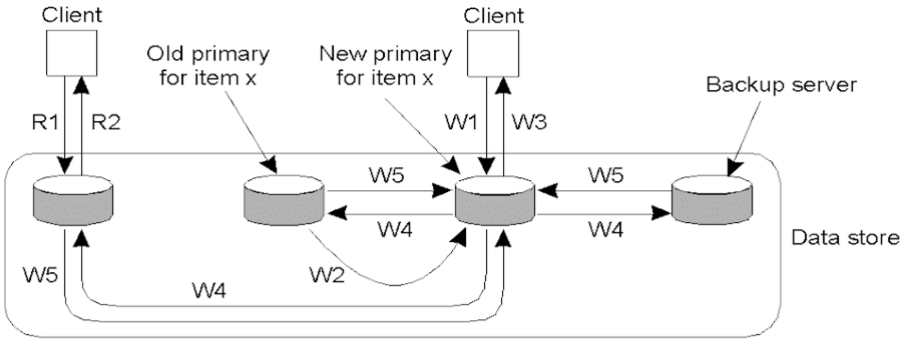
- ↓ Writes can take a long time
  - especially when blocking write
  - use non-blocking protocols
    - Primary acknowlege the write but crash before it could send writes to backups, the read-your-writes guarantee is broken
- ↑ Easy to implement **sequential consistency** (All processes agree on **some** global total ordering of events)
  - Primary send all writes to each replica **in the same order** via **Total FIFO-Order Atomic Multicast**
  - Implements **Strong consistency** if read requests are also forwarded to the primary (Updates should be **visible to all processes** at **the same time** $t \in [t_{\text{start}}, t_{\text{end}}]$)

Consistency Guarantee (Close to these but may not be exactly these)

- Strong Consistency: All reads (and writes) are directed to the primary
  - Every client always see the most up-to-date version
- Sequential Consistency: Primary update using **Total FIFO-Order Atomic Multicast**
- Eventual Consistency: use primary uses non-blocking update (acknowledge the client immediately, update in the background)
  - On crash another node is promoted as the primary. Do we still have this guarantee?

Primary-backup local-write protocols

- Primary migrates to the replica that is writing, successive writes are carried out locally, and then the replicas are updated using a non-blocking protocol



W1. Write request
W2. Move item x to new primary
W3. Acknowledge write completed
W4. Tell backups to update
W5. Acknowledge update

R1. Read request
R2. Response to read

Courtesy of [Tane06]

4.2.2 Primary-Backup Local-Write Protocol

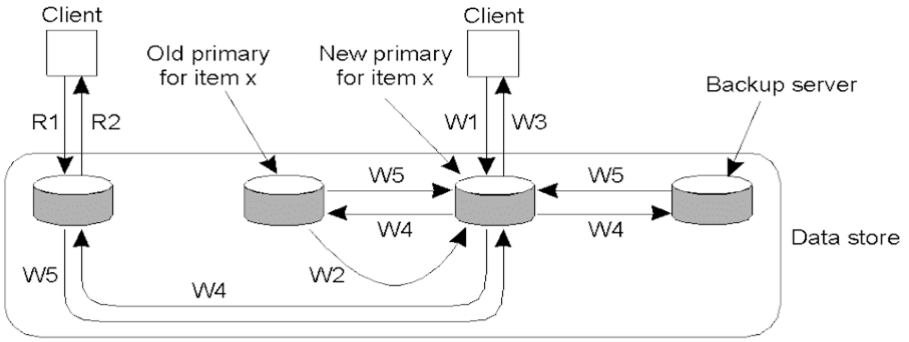
- Primary migrates to the replica that is writing
  - **NOTE** (Gemini): one primary for a specific **data item** at **any given time**
    - **Serialisation**: Only one process holds the primary copy before any writes can occur

  - **Global Order**
    - Writes only happen at current primary
    - Primary only migrates after completing/synchronising its state
      - Contradicts with the non-blocking protocol. Which one to use?
- Successive writes are carried out locally
- Replicas are updated using a non-blocking protocol
- For consecutive updates to the same object, we only propagate the last value (higher performance)
- ↓ False sharing for concurrent writes (primary jump back and forth between two clients' chosen replicas) ⇒ Use replicated-write protocols

Q: What consistency guarantee does this provide

Primary-backup local-write protocols

- Primary migrates to the replica that is writing, successive writes are carried out locally, and then the replicas are updated using a non-blocking protocol



Courtesy of [Tane06]

4.3 Replicated-Write Protocol

Writes can be carried out at **multiple replicas**

4.3.1 Active Replication

- **Each operation** is forwarded to **all replicas**
- Operations need to be carried out **in the same order everywhere**

Clients uses **Total FIFO-Order multicast** (not atomic?) to send operations to the group of replicas.
Consistency Guarantees

- **Sequential Consistency**: multicast **only write** operations
- **Strong Consistency**: multicast both **read and write** operations

4.3.2 Quorum-Based Protocols

Write to N_R replicas before writing
- Update replicas in quorum, assign **new version number**
- Remaining replicas are updated as a **background task**

Read N_R replicas before reading
- not all replicas in N_R needs to be up-to-date
- Any up-to-date replicas may be read
- Avoid Read-Write Conflict: $N_R + N_W > N \Rightarrow$ **Always exists** an up-to-data value
- Avoid Write-Write Conflict: $R_W > \frac{N}{2}$

Configurations
- $N_R = 1, N_W = N$: Optimised for read (ROWA: Read-One, Write-All)
- $N_R = N, W_W = 1$: Optimised for very fast write
- $N_R + N_W \leq N$: Eventual Consistency (May pick disjoint sets for reads and writes)
- $N_R + N_W > N$: Read-Write Conflict

5 Redis

- Data structures
  - Sets, Hashes, Sorted Hashes
  - Lists
  - Bitmaps
- Publisher/subscriber paradigm
- Allow writes only with attached replicas
- key expiration

5.1 Consistency Protocol

- Primary-Backup Remote-Write
  - On Redis, write operation on non-primary nodes are disabled (don't need to forward request to the primary, all requests should make to the primary node)
  - Read request goes to **any** of the replicas, which means that it may read most recent or older values (if the replica has not been updated yet)
  - Asynchronous
    - timeout
  - Non-Blocking
    - `min-replicas-to-write`: minimum replicas to write until the primary acknowledge the client (otherwise timeout and fail)

5.2 Fault Tolerance

- Master Failure: No updates allowed, system is still accessible, data read may be stale
- Some writes may be lost during the master election

Use `sentinal` nodes to elect new masters

`sentinal monitor <master-name> <master-ip> <master-port>` **`<quorum-size>`**
- Sentinel detect master's down (no heartbeat messages from the master)
  - `down-after-milliseconds`
- start fail-over mechanism
- A quorum of at least`<quorum-size>` nodes are required to declare the master is down
  - The `<quorum-size>` is **only** used to **detect master failure**

- Leader election requires a **majority** of the sentinel processes
  - **NOTE**: The majority means the majority in **total # of nodes**. If on internet partition, we still need $\lfloor \frac{N}{2} \rfloor + 1$ nodes to elect a master, where N is the **total # of nodes**, not the number of nodes in a partition

5.3 Commands

- Create redis cluster
  - Create master: `redis-server` (default port 6379)
  - Create replicas: `redis-server --replica-of <master-ip> <master-port>`

On-Disk Data Management

1 File System Properties

File System needs to solve two problems
- Naming: How do users locate their data
- Operations: How do users access their data

2 Naming

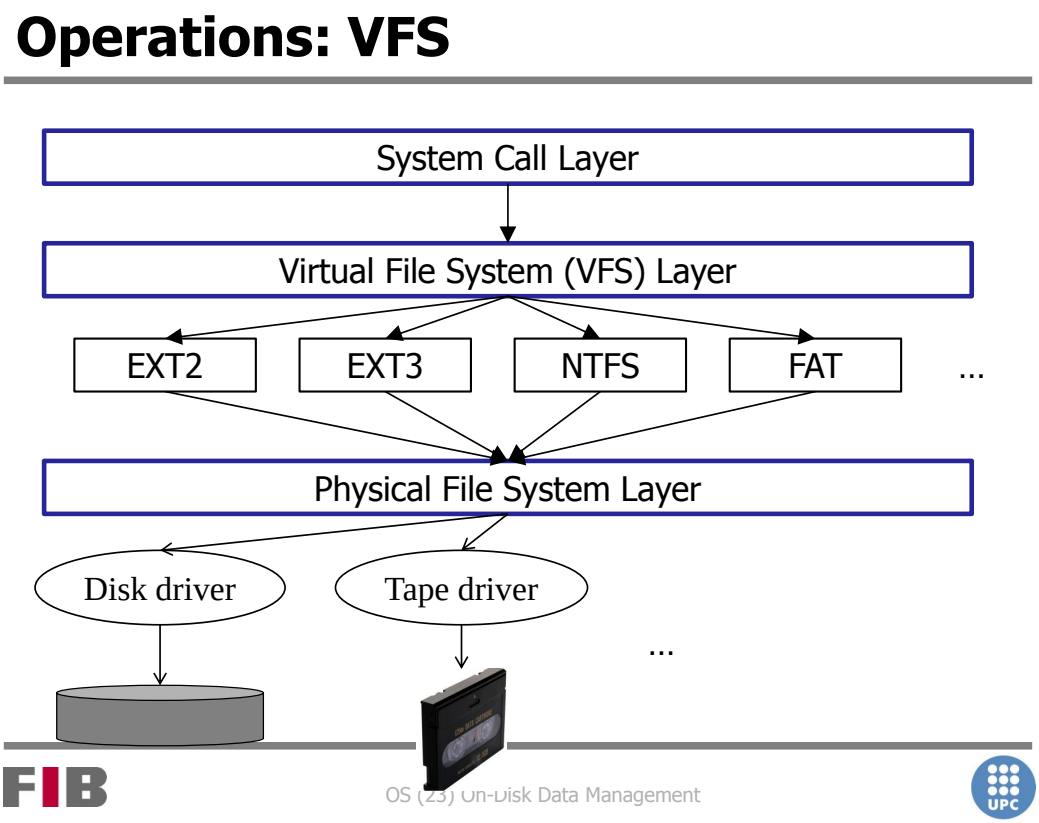
How do users store/locate their data ⇒ Identification, Naming **Name**: Map logical name and physical content
- User: map logical data objects to file names
- System: manages physical content
- filename → (i-node)number → blocks

Name hides implementation: how and where the file is stored on disk **Namespace**: Organisation of names
- Create names and organise names
- Represented as a labeled and directed graph with two types of nodes
  - Leaf Node: Entry, stores file content
  - Directory Node: Directory table of (edge-label, node-id)
- A naming path: $< \text{label}_1, ..., \text{label}_N >$
- Path name: Absolute path name, relative path name

Directories

3 Operations

Virtual File Systems: provides uniform and transparent access to files
- System call layer
  - `open`, `read`, `write`
- Virtual file system
  - Access to files independent of where and how they are stored
  - Translation file ↔ OS blocks identifiers
- Physical file system
  - Device specific code to read/write block content
  - e.g. Disk driver, tape driver



4 Distributed File Systems

4.1 Semantics for Shared Files in Distributed Systems

4.1.1 Sequential Semantics (UNIX)

Every operation on a file is instantly available to **all processes**

4.1.2 Session Semantics

No changes available to other processes until the file is **closed**

4.1.3 Immutable Files

No updates possible, use new versions of files. Simplifies sharing and replication

4.1.4 Transactions

All changes occur **atomically** (`BEGIN_TRANSACTION`, `END_TRANSACTION`)

4.2 Architectures

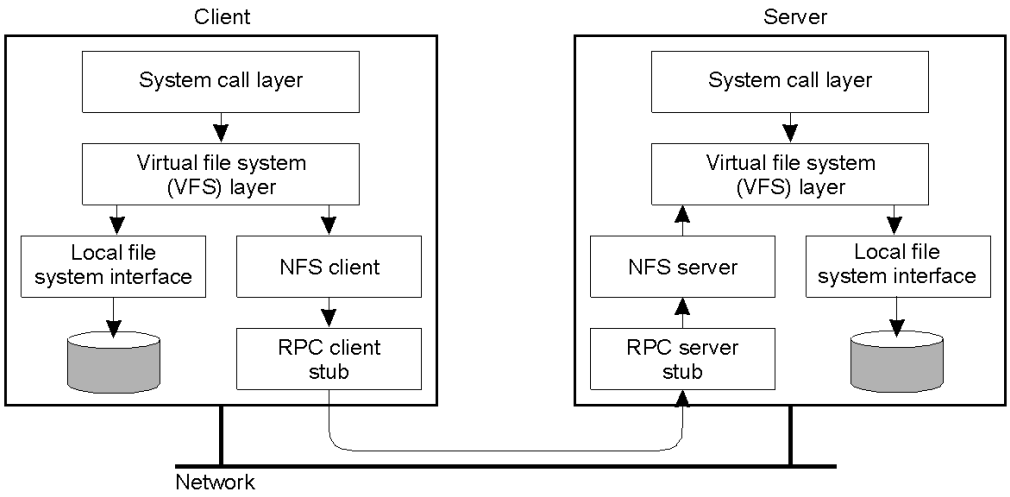
4.2.1 Client-Server Architecture

Make remote files appear as if they were stored on a user's local disk.
Separates data storage (server) from data consumption (client).
It uses network as a transparent bridge

4.2.1.1 Implementation

- Client uses FS calls locally
- Request is forwarded to the remote file server
- File Server access local disk
- Send back the response with the request data

NFS Architecture



Courtesy of [Tane06]

4.2.1.2 Client Side

Client mount remote directory locally

- **VFS layer**: Kernel-level abstraction that allows the OS to handle different file systems using a single unified interface
- **Client Module**: implements DFS protocol, converts local FS calls to network requests
- **Cache**: Client stores copies of recently accessed data in a local memory or disk cache (Example: Sun NFS v4)
  - Cache data obtained from **read** operations
    - Validate cache blocks before they are used
  - **Writes** carried out locally
    - Modified blocks are marked **dirty**
    - **Closing** the file implies **flushing** modifications to the server
  - Files can remain cached after closing it
    - Data must be **revalidated** when the file is reopened
    - Validation through timestamps (clock drift?)
  - Consistency problems

4.2.1.3 Server Side

- **Flat File Service**: Handles actual data blocks and performs operation on file contents

- **Directory Service**: Mapping between file names and unique identifiers (file IDs, i-nodes). Handles directory tree structures and metadata
- **File Exporter**: Interface listens for requests from clients and interact with the server's local FS to retrieve the data.

4.2.1.4 Access Models

4.2.1.4.1 Remote Access Model

- Clients are offered transparent access to the FS
  - RPC: allow the client to call a function on the server as if it were local

4.2.1.4.2 Upload/Download Model

- Client access locally to a file after downloading it from the server

4.2.1.5 Server Stateness

4.2.1.5.1 Stateful

- Store and manage files' state between operations
- Client requests → update the state
  - better performance

4.2.1.5.2 Stateless

- Nothing is stored at server side
- Client stores state
- Full requests from client with all required info
  - file, offset, bytes
  - worse performance

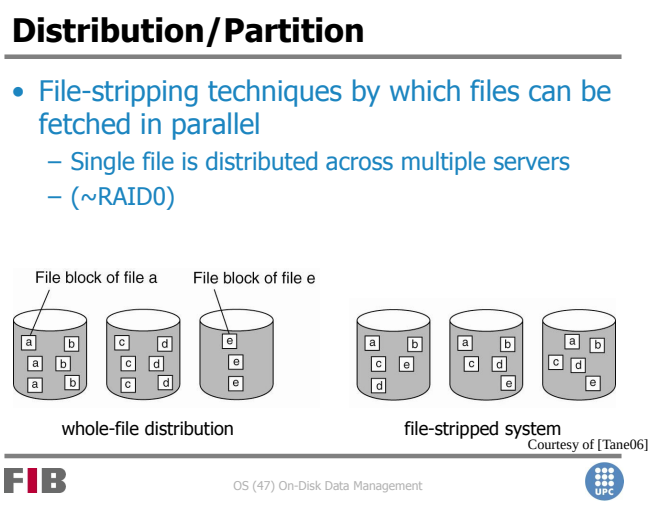
4.2.2 Cluster-Based Architecture

4.2.2.1 Replication

Replicate through multiple servers to allow single file to be accessed by multiple clients

4.2.2.2 Distribution (Partition)

File-stripping: distribute the blocks of a file across servers → files can be fetched in parallel.
Use metadata server and load balancer



4.2.2.2.1 Consistency Problems

Semantics

- **Immutable Files**: no algorithm needed
- **Sequential/UNIX Semantics**
  - (✗) Avoid replication: NOT viable
  - (✗) Avoid write caching: slow
  - (✗) Only cache on server: bottleneck
  - (✓) Coherence Algorithm
- **Session and Transaction**: Coherence Algorithm

4.3 IBM General Purpose File System (GPFS)

- Cluster-based Architecture
- Distributed Metadata
- No Repatition
- Consistency
  - UNIX/sequential consistency semantics
  - Byte-Range Locking: uses distributed token manager to lock a specific range of a file
    - e.g. Node A can write to the first 1MB of the file while node B can write to the second 1MB of the file simultanuously
  - Sequential access
    - Optimised for large sequential reads and writes
- Remote Access Model
  - Direct I/O: perform read/write on a mounted drive. Data is pulled in blocks only when required
  - Client Cache
    - Data Cache
    - Inode Cache
      - Internal file identifiers
    - File Attributes Cache
      - e.g. owner, permissions
  - Token-Based Consistency
- Stateful Server
  - System keeps

4.4 GFS

- Can allow multiple writers to append to different files simultaneously
- A file is accessible when there is metadata about this file
- A file can be partially accessible when all replicas for some chunks of the file fails/network partition
- Very high $\frac{\text{Read}}{\text{Write}}$ ratio

Features

- Client connects directly with chunk servers
  - Clients ask the master for the location of chunk servers (which servers I should contact)
  - Caches the location for a limited time
  - interacts with chunk servers directly
- No **data caching** at server/client: utilise system buffer cache
- Clients **Caches metadata**
- Atomic namespace mutation

4.4.1 Interface

Not POSIX compliant, No Kernel/VFS for accessing files User-level APIs to access files Implemented in user space using the native Linux FS Files are organised in directories Operations

- Basic: `create`, `delete`, `open`, `close`, `read`, `write`
- Snapshot
- Record Append
  - Allows multiple clients to append to the same data concurrently
  - Guarantees atomicity of each client's append
  - **Atomically at least once** guarantee

4.4.2 Cluster

The GFS has **one master** and **multiple chunk servers**

ONE master:

- Stores file system metadata (names, attributes)
- Maps **files** to **chunks**

Chunk Servers:

- Stores data: fixed-size chunks
- Chunks replicated on several systems

4.4.3 Single Master

NOTE

- All **metadata** are stored **in memory**
- **Namespace** and **file-to-chunk mapping** are also **kept persistent** by logging (**`operation log`**)
- Namespace mutations are **atomic**, guaranteed to have a **global total order** by the operation log

Maintains **all** file system **metadata**

- Namespace (PERSISTENT)
- access control info (PERSISTENT)
- `"Filename"` → `Chunks[]` mapping (PERSISTENT)
- Current locations of chunks (NOT-PERSISTENT)
  - Queries from the chunk servers
  - Chunk servers have the final say on which chunk to have on its disk

Manages

- Chunk leases (locks)
  - Clients needs to ask permission from the master
- Garbage Collection (freeing unused chunks)
- Chunk Migration (copying/moving chunks)

Replication The master **replicates its data** for fault tolerance

Communication
The master periodically communicates with all chunk servers

- Via **Heartbeat** messages
- To get states and send commands

4.4.4 Chunk Servers

- **Chunk size** = 64MB by default
- **Checksum**: Stores **in-memory** and **on disk** a 32-bit checksum with **each chunk**
- **Chunk handle**: unique 64-bit number

  - **assigned by the master** on chunk **creation**
- **Chunk as Linux Files**: Chunk servers store chunks on local disk as Linux files
  - usually under `/tmp`
  - total storage of chunks can be estimated by the total space of the `/` directory
- **Chunks Replicated**: Each chunk replicated on multiple chunk servers
  - By default, 3 replicas
  - Popular files may need more replicas to avoid hotspots

4.4.5 Read Path

1. Contact the master
2. Get the list of chunk handles and file's metadata
3. Get location of each of the chunk handles (replicated info)
4. Contact **any** available chunk server

4.4.6 Write Path

1. Master grants a **chunk lease** to one of the replicas
   - The master informs the primary replica of its primary selection
   - This replica will be the **primary replica**
   - The master increases the **chunk version number** and informs replicas
     - The replicas must record this new version number in **persistant storage** before master can reply to the client
   - Client should contact the **master** only when the primary replica
     - Is not reachable, or
     - Replies that the client no longer has the lease
2. **Control Phase**: send data (don't write, cached)
   1. Client given a list of replicas
   2. Client writes to the closest replica
   3. The replica **forwards** to another chunk server
   4. Chunk servers store this data in a cache
3. **Data Phase**: Write data
   1. Client waits for replicas to acknowledge receiving data
   2. **Send write request to the primary replica**
      - The request identifies the data pushed to all the replicas
   3. The **primary** is responsible of **serialisation of writes**
      - Assign consecutive **serial numbers** to all writes
      - Serial numbers are used to detect stale values after a partition
      - When multiple clients want to write to the same offset
        - They are assigned to the same primary replica
        - The primary replica determines the write order
        - Regular writes may be undefined, use atomic append
   4. Once all acknowledgements have been received, the primary acknowledges the client
   5. If any error from the replicas, the client code will try to retry (The state is inconsistent)

4.5 HDFS

TODO

5 Distributed Databases

Traditional Databases

- Data stored in (structured) tables
- Each row (data item) in a table has a unique Primary Key
- Queried using SQL
- Support joints

Today
- Large and unstructured data (Available > Consistent)
- Lots of random reads & writes
- Sometimes huge number of writes
- **Foreign Keys rarely used, infrequent joins**

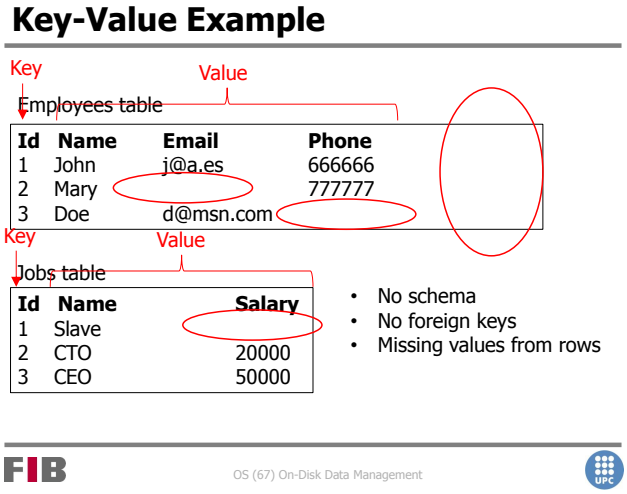
Needs
- Speed
- Avoid Single Point of Failure (SPoF)
- Low Total Cost of Operation (TCO)
- Fewer system administrators
- Incremental Scalability, **scale OUT**

5.1 Scale Up/Out

- Scale up: Grow cluster capability with more **powerful machines**
  - Traditional, not cost-effective
  - Need to replace machines often
- Scale Out: **Incrementally** grow cluster capability by adding more **COTS machines** (Components Off the Shelf)
  - Cheaper
  - Over a long duration, phase in faster machines as we phase out few older ones

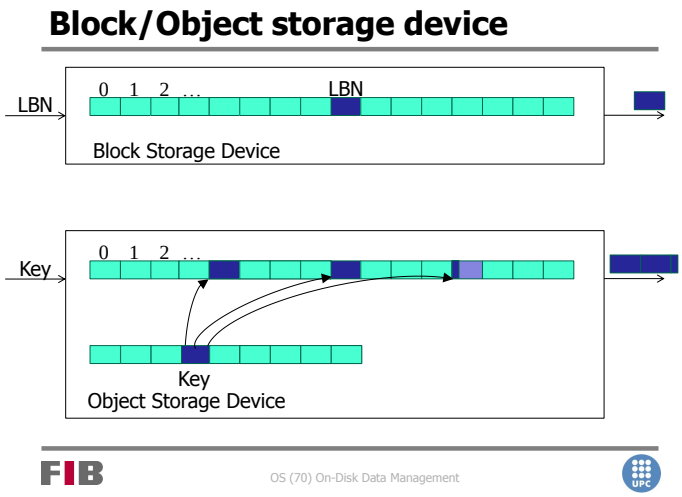
5.2 Key-Value (NoSQL) Data Model

APIs: `get(key)`, `put(key, value)`
Tables (Column Families, Table, Collection)
- Like relational DBMS, but **not structures** (**missing columns** from rows)
- Supports for joins or foreign keys are **slow** and not used
- Can have indices



5.3 Storing Data

Block-based FS stores structured (fix-sized) data well, but not for unstructured (variable-sized) data.

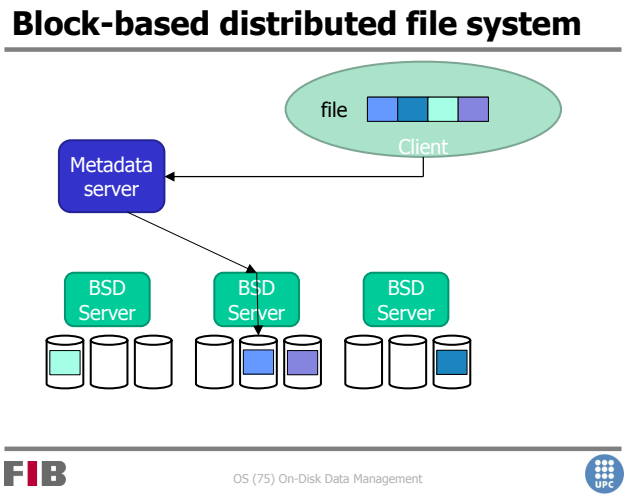


5.3.1 Block Storage Device

- Data stored in fix-sized blocks
- Identified by Logic Block Number (LBN), a positive integer starting from 0
- LBN used for data retrival

Responsibilities

- **Metadata Server**
  - Metadata
  - which **disk** and **block** a data is stored



5.3.1.1 File Creation

1. Client → Metadata Server (create)
   - Server allocates an **i-node**?

5.3.1.2 File Access

1. Client → Metadata Server

- `Filename`, `Offset`

2. Metadata Server → Client
   - `(Disk#, Block#)`
   - OR: the block

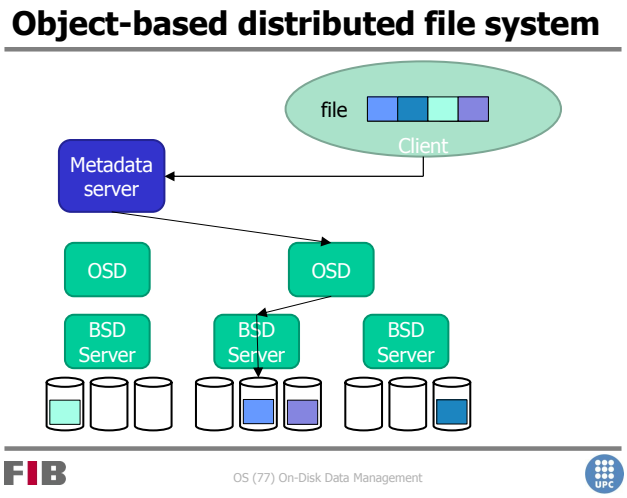
5.3.2 Object Storage Device (OSD)

Decouples metadata (naming) from data location Data can be moved inside Object Storage Devices without notification

- Data is a single object (Key, Value)
- Key: Positive integer
- Attributes
- Value: Arbutrary size

Responsibilities Clients access directly both **Metadata Server** and the **OSD**

- **Metadata Server**: Metadata about the object (file → object ID)
- **Object Storage Device**: which disk and block the data is located (Object ID → i-node)



1. Client: `path/to/file`
2. Metadata Server: `objectID: 42`
3. Object Storage Device: `inode`
4. Block Storage Device: Retrieve data

5.3.2.1 File Creation

1. Client → Metadata Server
   - Server allocates **i-node**
2. Metadata Server → Object Storage Device
   - Create object for this file

5.3.2.2 File Access

1. Client → Metadata Server
   - `path/to/file`
2. Metadata server → Client
   - List of Object Storage Devices
3. Client → Object Storage Devices
   - `File`, `Offset`

4. Object Storage Device → Client
   - Data

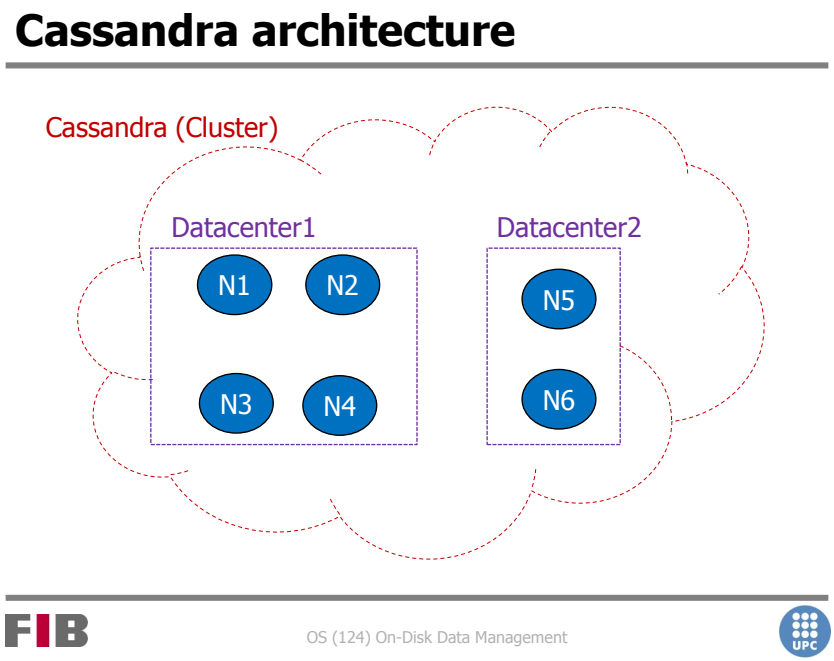
5.4 Cassandra

- Peer-to-peer distributed system across homogeneous nodes

Main Elements

- **Nodes**: where data is stored
- **Datacentre**: collection of nodes
  - **Granularity** used by **replication**
  - Different workload can use separate datacentres
  - Datacentres must have **never span physical locations**
- **Cluster**: 1 or more datacentres

Cassandra is aware of network topology



5.4.1 Replication

Replication Factor: total number of replicas across the cluster

- `replication_factor` copies of each row, each copy on a **different node**

Replica Placement

- Partitioner: determines
  - which node will receive the **first replica** of a piece of data
    - determined using consistent hashing
- Replica Placement Strategy
  - how to distribute other replicas across other nodes
  - `SimpleStrategy`: Move clockwise along the ring, pick RF − 1 nodes
    - From the slide: Single datacentre and 1 rack?
  - `NetworkTopologyStrategy`
    - Multi-DC Aware: Specify different RF for each datacentre

  - Rack Aware: Within a datacentre, try to place replicas on **different racks**

5.4.2 Consistency

Per-query tunable consistency: how many nodes should response before an operation is considered successful

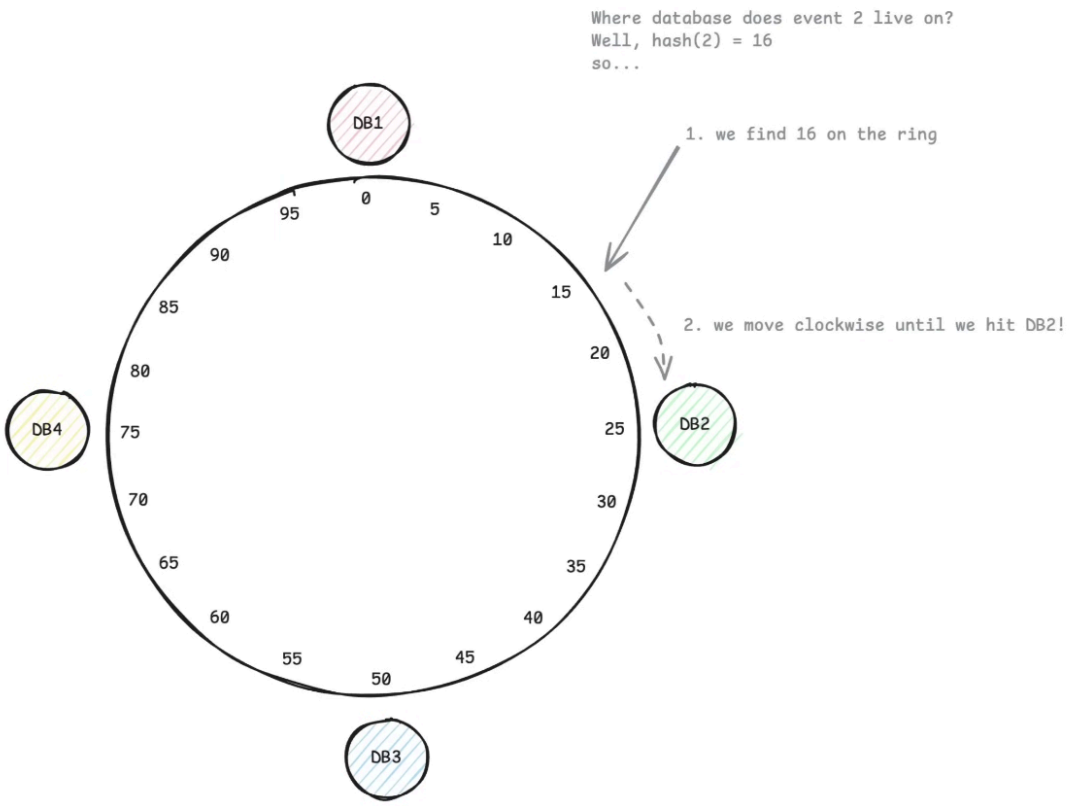
- **ANY**: At least one node
  - just the coordinator is okay, doesn't need to be a replica
- **ONE**: At least one replica (**any** node of the cluster)
- **LOCAL_ONE**: one replica from the **same** node as the coordinator
- **ALL**: All replicas
- **QUORUM**: Waits for the majority of replica nodes in
  - **QUORUM**: response from **any** nodes in the cluster, the majority of the sum of RFs in all nodes
    - $\left\lfloor \frac{\sum_{i \in \text{DC}} (\text{RF}_i)}{2} \right\rfloor + 1$
  - **EACH_QUORUM** (Only for writes): In **each** datacentre, require a majority
    - $\forall i \in \text{DC} : \left\lfloor \frac{\text{RF}_i}{2} \right\rfloor + 1$
  - **LOCAL_QUORUM**: majority in the **same** datacentre as the coordinator

Replicas mean the number of replicas of this specific data (`replication_factor`)

5.4.3 Consistent Hashing

- Use a consistent key-space: a ring
- Data has a partition key
- Each partition key is hashed into a ring (`hash(partitionkey)`)
  - the hashed number is a 64-bit integer
- Each node in the cluster has a range of data based on hash value
- Cassandra place data on each node according to
  - Partition key
  - The responsible range of node

Walk clockwise to find the node



5.4.4 Scalability

5.4.4.1 Gossip Protocol

Nodes are "equal", exchange messages using a **gossip protocol**

- alive nodes, dead nodes
- partition ranges
- schema versions (UUID)
- used disk spaces
- RPC IP addresss

Each node has its own view of the cluster, and each node can be the "coordinator" for a specific request

5.4.4.2 Bootstrap (Add Node)

To add a new node, specify

- `cluster_name`: the cluster to add the node (must match existing names)
- `seeds`: A subset of existing nodes' IP addresses
  - Used to contact when first joined in to gain full topology of the cluster
- `listen_addr`: Internal IP of the new node

Contact Seed Nodes

- Send gossip message to **one of** the **seed nodes** to get the topology of the cluster
- Start gossip with **every node**

Token Selection

- Default: select randomly `num_tokens` (usually 256) across the ring
- Allocation Algorithm: relieve the heaviest-loaded existing nodes

Join and Streaming

- Once the tokens are selected, the node enters **JOIN** state
- Contact the existing node that currently hold those ranges
- Existing node begins sending (streaming) the relevant SSTable files to the new node over the network
- During this process, the new node **accepts writes** (so it don't miss new data), but does **NOT serve read** yet because its data is still incomplete

use `nodetool netstats` to check

Transition to NORMAL
- Once the streaming is 100% complete, the node update its status in **Gossip** to **NORMAL** (UP/NORMAL)
- The coordinator nodes access the cluster are notified
- Read requests will now be routed to this node

Cleanup
- The **old nodes** still have copies of that data that they no longer **own**
- Manually run `nodetool cleanup` on the neighbouring nodes
- Trigger a process where the old nodes delete the data range they no longer responsible for, freeing up disk space

5.4.4.3 Removing Nodes
- Use `nodetool` to remove node
- Tokens are reassigned automatically

Decommissioning
- `nodetool decommission`
- When node is **alive and healthy**
- The leaving node will identify which other nodes will take over its token ranges
- Physically streams its data to those new owners
- Enters the **LEAVING** state using gossip
- After streaming, sends the final "goodbye" message via gossip and shuts itself down

Remove Node
- `nodetool removenode` with the `Host ID` of the dead node
- When the node is physically failed or unreachable
- The remaining replicas for the token ranges will stream data to the node that are picking up the slack
- Cluster marks the dead node's ID as **REMOVED**

Assassinate
- `nodetool assasinate <IP>`
- Forcefully tell everyone to immediately forget about the target ID
- Does **NOT** stream the data, lost replica

Final Cleanup run `nodetool cleanup`

5.4.5 Write Path
Client
- Client can contact **any** node as the **coordinator** for this request
  - Each node has a full view of the cluster (which nodes contains the required data) thanks to the gossip protocol
  - Coordinator's responsibility

  - Identify which replicas hold the data based on partition key
  - Manage communication between those replicas

Coordinator

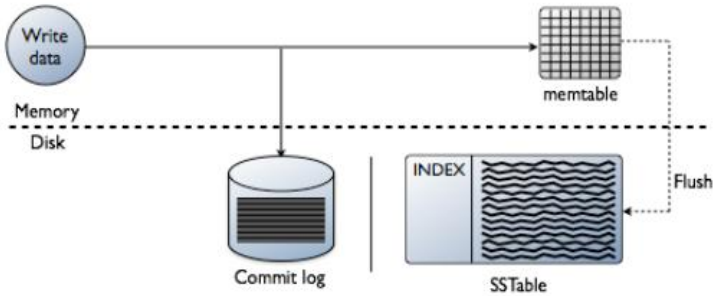
- Coordinator send writes to **all** replicas

Replicas (receiving write requests)

- Replica writes to the write-ahead commit log
- Replica writes to the memtable (sorted map of recently written rows)
- Return to the client to acknowledge the write
- In the background, Each time the memory structure is full
  - flush it flush to disk into the **SSTable** (Sorted String Table)
  - SSTable is **immutable**, append-only
  - New writes creates new SSTable
  - SSTable are merged through **Compaction** (compact old rows)

Cassandra: How is data written?

- Logging data in the *commit log*
- Writing data to *memtable*
- Flushing data from memtable
- Storing data on disks in *SSTables*
  - *Sorted String Table*
  - immutable data file
  - append only and stored on disk sequentially



FIB OS (134) On-Disk Data Management UPC

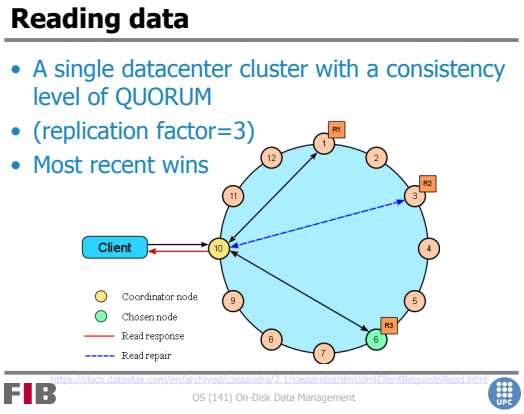
5.5 Read Path

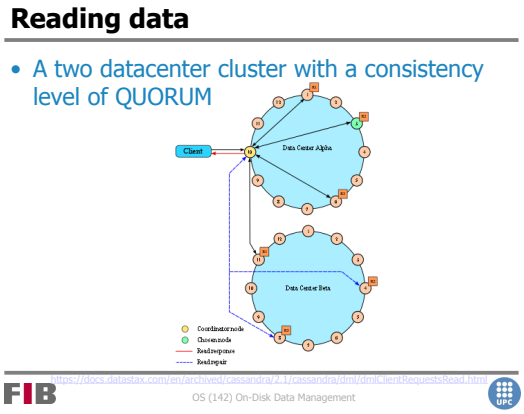
Coordinator

1. Send a **full read request** to the replica with **highest performance** (determined by dynamic snitching)
2. Send **digest request** to remaining (specified by the consistency level, not necessarily all) nodes
   - Digest request response: MD5 (hash) value of the data
3. If the data's hash matches all digest data: Consistent, replay client immediately
4. If exists mistaching: Perform **Read Repair**
   - send **full read** to **ALL** replicas

  - determine the latest version based on **timestamp**
  - update out-of-data nodes
- Coordinator waits for replies (according to consistency level), and then return

Replicas (receiving read requests)
- Checks **memtable** (in-memory buffer), if data's there, return immediately
-





5.5.1 Key-Space

Use `IF NOT EXISTS` because multiple clients may create the same **Key-Space** or **Table** at the same time Class:
Replication Factor:

```
CREEATE KEYSPACE IF NOT EXISTS store WITH REPLICATION = {
  'class': 'SimpleStrategy',
  'replication_factor': 1
};
```

5.5.2 Tables

In Cassandra, tables are called **Column Families**.
When creating tables, we use the name `<key-space-name>.<table-name>` to denote Table $\in$ KeySpace. Otherwise, use `use <key-space-name>`
- Primary Key = partition-key : clustering-key where
  - `partition-key` determines which node to store the data
  - `clustering-key` determines the row within a partition

```
CREATE TABLE IF NOT EXISTS store.shopping_cart (
  userid                TEXT PRIMARY KEY,
  item_count            INT,
  last_update_timestamp TIMESTAMP
);
```

Energy Management

Energy Consumption Accounting
- **Power Metering**: Measure the actual power consumption by instrumenting the hardware
- **Power Modeling**: Estimate power consumption by means of corelated indirect evidences
  - Performance monitoring counters

- Resource utilisation metrics

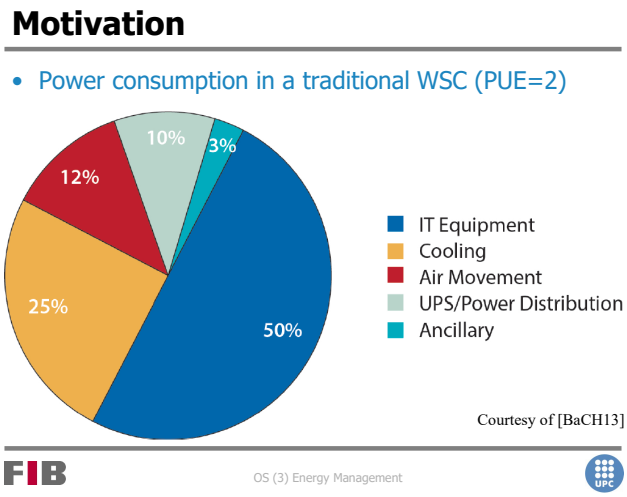
1 Metrics

1.1 Power Usage Efficiency (PUE)

Ratio multiplier to convert from **Total Energy Consumption** to **IT Equipment Consumption**

- Hyperscale WSCs (e.g. Google): 1.13
- Traditional data centres: 1.6 — 2.35

$$\text{PUE} = \frac{E_{\text{Total}}}{E_{\text{IT}}}$$



- Not all PUE measurements include the same overheads ⇒ Adhere to Green Grid guidelines in measuring and reporting
- Instantaneous PUEs differ from average PUEs ⇒ Annual averages from data aggregates
- Some PUEs are based on optimal operating environment ⇒ Use real-time measurement

1.2 Server Power Usage Efficiency (SPUE)

Ratio multiplier to convert from **Server Input Power** to **Useful Power**

$$\text{SPUE} = \frac{E_{\text{In}}}{E_{\text{Useful}}}$$

- **Useful Power**: power consumed **directly** involved in computation
  - Motherboard, disks, CPUs, DRAMs, etc
- **Unuseful Power**: Cooling fans, power supply, voltage regulator modules, etc.
- Google's SPUE $\sim$ 1.1

1.3 Total Power Usage Efficiency (TPUE)

$$\text{TPUE} = \text{PUE} \times \text{SPUE}$$

1.4 Energy Efficiency of WSCs

Measures energy used by a WSC to run a **particular workload**

$$\text{Eff} = \frac{E_{\text{Computation}}}{E_{\text{Total}}} = \frac{1}{\text{PUE}} \times \frac{1}{\text{SPUE}} \times \frac{E_{\text{Computation}}}{E_{\text{ElectricComponents}}}$$

1.5 Data Centre Energy Productivity (DCeP)

Measures how the energy delivered to electronic components is actually converted into useful work
- Typically, benchmarks have been used (e.g. SPECpower)

In exams, it is defined as

$$\text{DCeP} = \frac{\sum_{\text{U}=0}^{100} \text{TotalWork}(U)}{\sum_{\text{U}=0}^{100} \text{TotalEnergy}(U)} \left[\text{in } \frac{\#\text{Oprations}}{\text{Watt}}\right] \tag{10}$$

In which we profile each utilisation

2 Power Metering

Power-Energy Relation (Energy is power over time)

$$E = T \cdot P \tag{11}$$

Units regarding energy:

$$1 \text{ Wh} = \underbrace{1 \text{ W}}_{P} \times \underbrace{3600 \text{ sec}}_{t} = 3600 \text{ J} \tag{12}$$

Frequency-Time Relation (Lower frequency, higher execution time)

$$T_1 \cdot f_1 = T_2 \cdot f_2 \tag{13}$$

Total Work is defined as

$$\text{TotalWork} = \text{Performance} \times \text{Time} \times \#\text{Hosts} \tag{14}$$

Total Energy is defined as

$$\text{TotalEnergy} = E \times \#\text{Hosts} \tag{15}$$

2.1 Integrated Metering on Hardware

Firmware offers API to query power usage
- Add specific sensors (e.g. Dell PowerEdge, HP ProLiant)
- Use software power models (Intel Running Average Power Limit, RAPL)
- ↑ Accurate power for individual components
- ↓ Architecture dependent
- ↓ Intrusive method (metering procedure itself consumes power)

2.2 External Meter

- General purpose meters
  - not appropriate for large-scale deployments)
  - e.g. PLOGG, Kill-A-Watt
- Intelligent Power Distribution Units (PDUs)
  - e.g. Raritan Dominion PX, HP Intelligent PDU
- ↑ Architecture independent
- ↑ Unobstructive external measurements
- ↓ Can measure only system-level power
- ↓ Less accurate measurements

3 Power Modeling

Used to account for energy consumption when

- Direct measurement is not feasible
- To access the power consumption of **individual software components**
  - e.g. processes, virtual machine
- Forecast the power consumption in the future or in hypothetical scenario

Aspects

- Application workloads
- Server power load and energy consumption
- Server temperature and heat distribution
- Cooling and air flows
- Cost (operational and capital expenses)

Scales

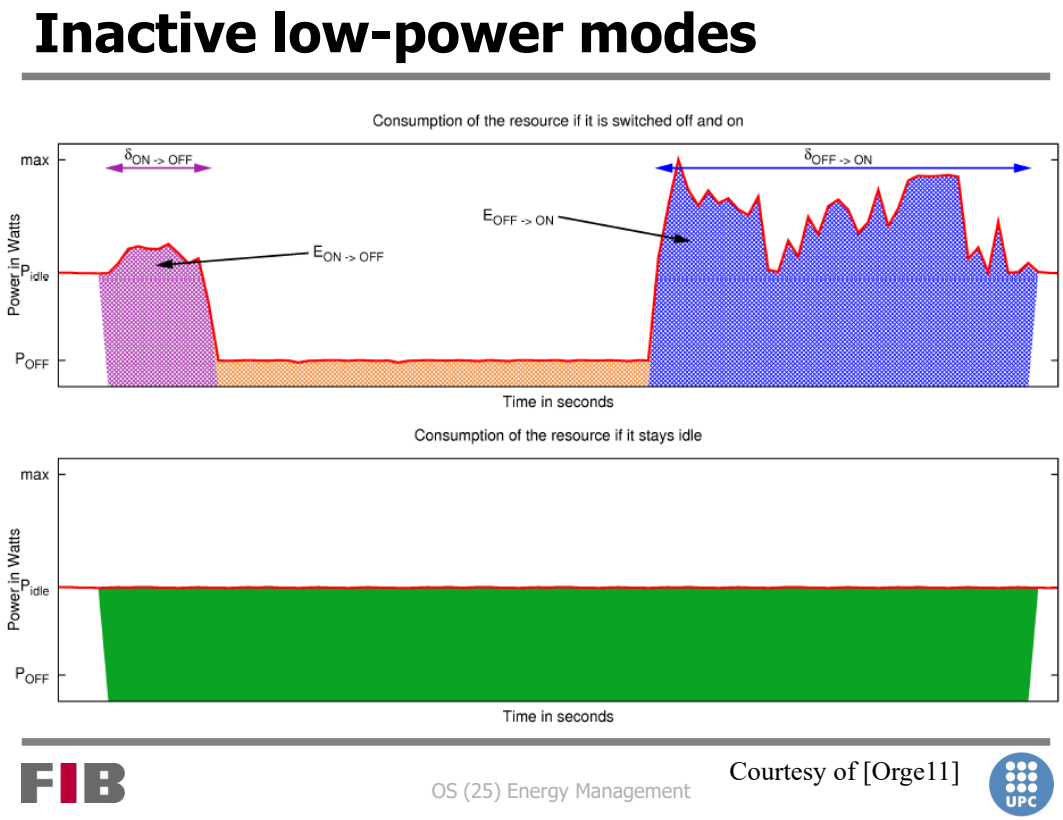
- Time Scales: ms, s, min, hr
- Special Scales: components, servers, racks, WSCs

4 Efficiency of IT Equipment

4.1 Low-Power Modes

4.1.1 Inactive Low-Power Modes

Turn off hosts when they are idle and turn them on when they are needed again



Here, if we want to compare the **inactive** and **active** power modes, we should limit

$$\delta_{\text{idle}}^{\text{active}} = \delta_{\text{ON}\to\text{OFF}}^{\text{inactive}} + \delta_{\text{OFF}}^{\text{active}} + T_{\text{OFF}\to\text{ON}}^{\text{active}} \tag{16}$$

Otherwise, we would have delayed the original schedule in the inactive mode, hence the comparison isn't fiar.
Hence

$$P_{\text{idle}} \cdot \delta_{\text{idle}} > E_{\text{ON}\to\text{OFF}} + E_{\text{OFF}\to\text{ON}} + (P_{\text{OFF}} \cdot (\delta_{\text{idle}} - \delta_{\text{ON}\to\text{OFF}} - \delta_{\text{ON}\to\text{OFF}})) \tag{17}$$

$$\Rightarrow \delta_{\text{idle}} > \frac{E_{\text{ON}\to\text{OFF}} + E_{\text{OFF}\to\text{ON}} - P_{\text{OFF}} \cdot (\delta_{\text{ON}\to\text{OFF}} + \delta_{\text{ON}\to\text{OFF}})}{P_{\text{idle}} - P_{\text{OFF}}} \tag{18}$$

4.1.2 Active Low-Power Modes

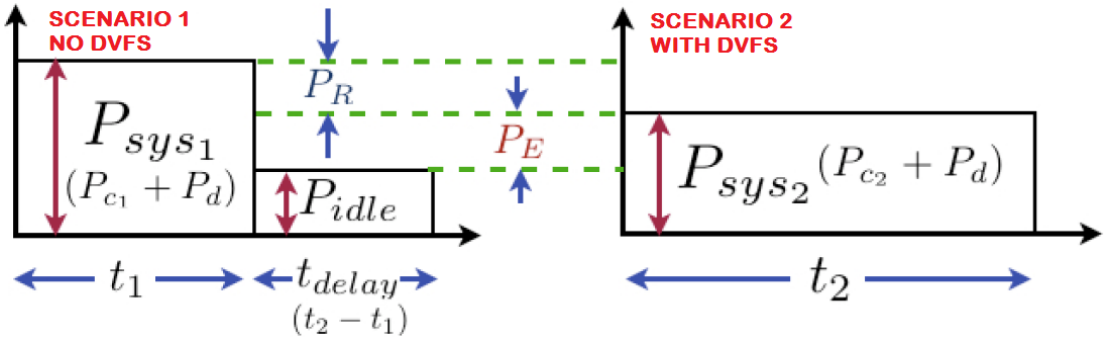
Better for **memory bounded** tasks (less effected by the frequency)
- Keep the host active but lower the CPU voltage/frequency to save power
- Decreasing frequency increases the execution time

Dynamic Voltage and Frequency Scaling (DVFS)
- Allow changing dynamically the operating voltage/frequency of the processor

Active low-power modes

- Only the time spent in the CPU and the CPU power are sensitive to DVFS
  - $t_2 = t_{c_2}(\uparrow) + \boldsymbol{t_d}$
  - $P_{sys_2} = P_{c_2}(\downarrow) + \boldsymbol{P_d}$

t_d: time in other resources
P_d: power of other resources
t_c: CPU time
P_c: CPU power



Observation:

$$P \sim \alpha C_L V_{\text{dd}}^2 f \tag{19}$$

Execution Time
Total execution time consists of **CPU** time t_c and **Device** time t_d

$$t = t_c + t_d \tag{20}$$

Resource Usage Percentage (Workload): helps to divide execution time $R_c + R_d = 1$, hence

$$\begin{aligned} t_c &= t \cdot R_c \\ t_d &= t \cdot R_d \end{aligned} \tag{21}$$

Since only **time** spent in the **CPU** and **CPU power** are sensitive to DVFS

$$t_{d_1} = t_{d_2} = t_d \tag{22}$$

Given frequency ratio $\frac{f_1}{f_2} > 1$, and since frequency **only** influences **CPU time**,

$$t_{c_1} \cdot f_1 = t_{c_2} = f_2 \tag{23}$$

Hence

$$\begin{cases} t_1 = t_{c_1} + t_d \\ t_2 = \underbrace{t_{c_1} \cdot \frac{f_1}{f_2}}_{t_{c_2}} + t_d \end{cases} \tag{24}$$

We denote the idle time (for non-DVFS configuration) as the waiting time

$$\begin{aligned} t_{\text{idle}} &= t_2 - t_1 \\ \Rightarrow t_{\text{idle}} &= t_{c_1} - t_{c_2} \end{aligned} \tag{25}$$

Power
Static power P_{idle} always consumes energy, and the current power is the sum of static and dynamic power

$$P = P_{\text{dynamic}} + P_{\text{static}} \tag{26}$$

hence

$$P = \underbrace{P_c + P_d}_{P_{\text{dynamic}}} + \underbrace{P_{\text{idle}}}_{P_{\text{static}}} \tag{27}$$

Note: If we divide the power and time as **CPU** and **Device**, we should calculate their power consumption separately

$$E = \underbrace{P_c \cdot t_c}_{\text{CPU Consumption}} + \underbrace{P_d \cdot t_d}_{\text{Device Consumption}} \tag{28}$$

We **CANNOT** calculate using

$$E \neq (P_c + P_d) \cdot (t_c + t_d) = \underbrace{P_c \cdot t_c + P_d \cdot t_d}_{\text{Correct } E} + \underbrace{P_c \cdot t_d + P_d \cdot t_c}_{\text{WRONG!}} \tag{29}$$

4.1.3 Non-Active and Active Power Saving Mode

For the **non-active configuration**, the total energy consumed should be

$$E_1 = \underbrace{\underbrace{\left(P_{c_1} + P_{\text{idle}}\right) \cdot t_{c_1}}_{\text{CPU consumption}} + \underbrace{(P_d + P_{\text{idle}}) \cdot t_d}_{\text{Device Consumption}}}_{\text{Full Frequency Consumption}} + \underbrace{P_{\text{idle}} \cdot t_{\text{idle}}}_{\text{Idle Consumption}} \tag{30}$$

For the **DVFS configuration**, the total energy consumption should be

$$E_2 = \underbrace{\underbrace{\left(P_{c_2} + P_{\text{idle}}\right) \cdot t_{c_2}}_{\text{CPU Consumption}} + \underbrace{(P_d + P_{\text{idle}}) \cdot t_d}_{\text{Device Consumption}}}_{\text{Decreased-Frequency Consumption}} \tag{31}$$

To be able to compare, we could (in theory) simplify to

$$E_2 - E_1 = P_{c_2} t_{c_2} - P_{c_1} t_{c_1} ??? \tag{32}$$

4.1.4 Default and Active Power Saving Mode

Power Measurement

$$\begin{aligned} P_2 &= \left(R_c \cdot \underbrace{P_1 \frac{f_2}{f_1} \frac{V_2^2}{V_1^2}}_{\text{CPU Power}} + R_d \cdot \underbrace{P_1}_{\text{Device Power}} \right) \\ &= \left(R_c \frac{f_2}{f_1} \frac{V_2^2}{V_1^2} + R_d \right) P_1 \end{aligned} \tag{33}$$

Work Measurement

We are given the performance perf_1 (req/sec) of the default configuration, our new configuration has the performance of

$$\begin{aligned} \text{perf}_2 &= \left(R_c \cdot \underbrace{\frac{f_2}{f_1} \text{perf}_1}_{\text{CPU Perf}} + R_d \cdot \underbrace{\text{perf}_1}_{\text{Device Perf}} \right) \\ &= \left(R_c \frac{f_2}{f_1} + R_d \right) \cdot \text{perf}_1 \end{aligned} \tag{34}$$

where $R_c + R_d = 1$

We can find the amount of work (instruction execution) done for both configurations

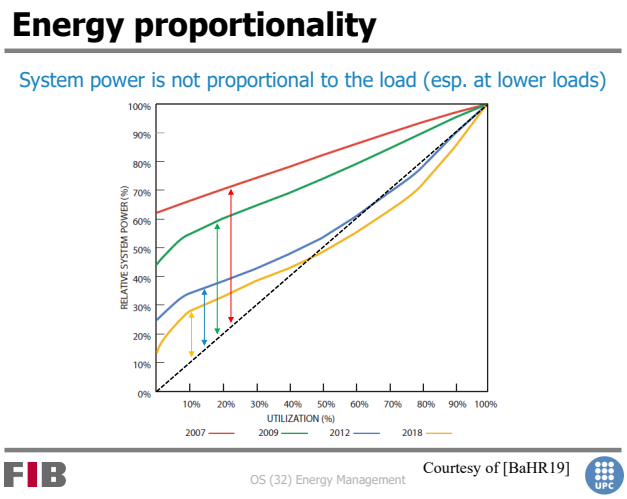
$$\begin{aligned} W_1 &= \text{perf}_1 \cdot T \cdot N_{\text{hosts}} \\ W_2 &= \text{perf}_2 \cdot T \cdot N_{\text{hosts}} \\ &= \left(R_c \cdot \frac{f_2}{f_1} + R_d \right) \cdot \text{prof}_1 \cdot T \cdot N_{\text{hosts}} \end{aligned} \tag{35}$$

5 Resource Allocation and Scheduling

Nonproportional energy consumption to load

Adapt policy to optimise energy efficiency
- When to run each task
- How many PEs to assign to each task
- Which PEs to assign to each task
- **(NEW)** What power state to set for each PE

Virtualisation technology
- VM consolidation
- Live MV migration
- VM allocation



5.1 VM Consolidation

- Run the hosts at **higher utilisations** closer to its **most energy-efficient operation point**
  - Live VM migration supports dynamic consolidation
- Reduce the number of hosts needed to serve a given workload
  - inactive low-power modes

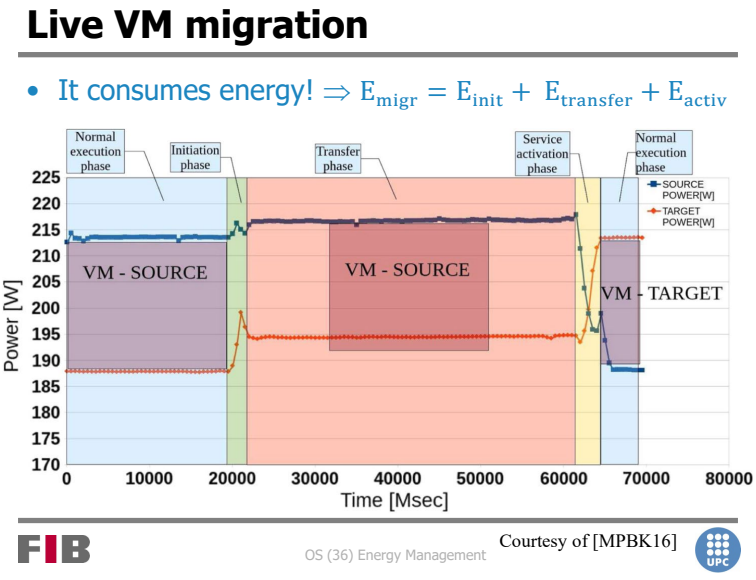
VMs in the same physical host may suffer **resource contention (interference)**

- → Use **contention-aware scheduling**

5.2 Live VM Migration

Consumes energy

$$E_{\text{migration}} = E_{\text{init}} + E_{\text{transfer}} + E_{\text{activation}} \tag{36}$$



(Most) Impacting factors on the energy $E_{\text{migration}}$

- CPU utilisation of the source and target hosts

  - Utilisation of each VM
  - Utilisation of the hypervisor
  - Utilisation added by the migration (VM suspension/restart, state transfer)
- Memory allocation to the migrating VM
- Memory dirty rate of the migrating VM
- Network of the bandwidth between the source and target for transferring the state of the VM

5.3 VM Allocation

Packing problem, consider more challenges
- Power consumption of the host regarding the load is non-proportional
- Host can have heterogeneous power consumption profiles regarding the load
- Migration has an energy cost

6 WSCs Ecosystems

Select the most appropriate WSC to deploy every workload

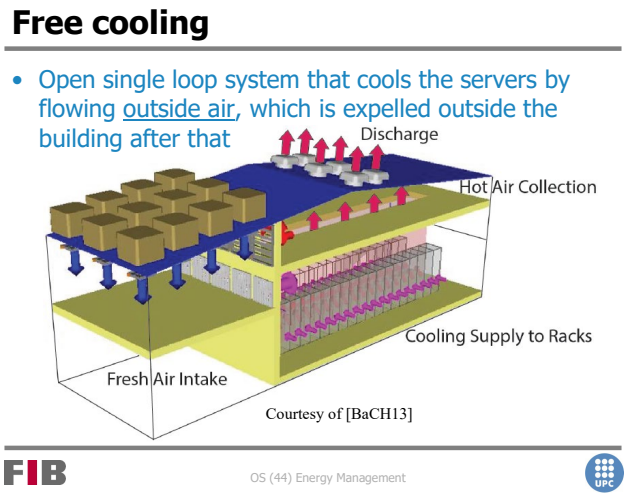
Follow the Moon: deploy workloads by following the night-time
- ↑ **Easier Workload Allocation**: computers in an off-peak demand interval
- ↑ **Reduced Costs**: cheaper electricity
- ↑ **Free Cooling**: Lower outside temperature
- ↓ **Distant Geographic Location**
  - Increased performance and energy cost for VM migration
  - Increased communication latency
- ↓ Independent administrative domains involved in the ecosystem
  - Conflicting goals among WSCs
  - Lack of information about the energy consumption and energy mix of other WSCs

7 Cooling Subsystem

Use some hierarchy of loop systems
Each loop brings in a cold medium (such as air or water) that absorbs heat via a heat exchange process and is subsequently cooled back down.

Open Loop: The system replace the ongoing warn medium with cool supply from the **outside**
Closed Loop: The system recirculate the same medium repeatedly. It transfer the heat to the **upper loop** through a **heat exchanger**

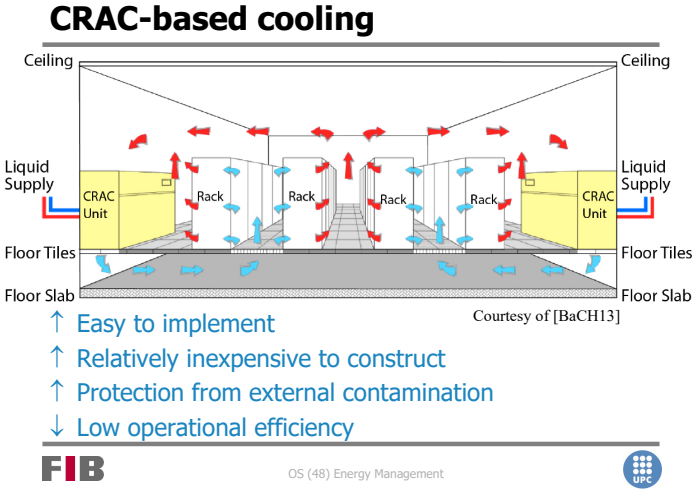
7.1 Free Cooling



Open single loop system that cools several servers by flowing **outside air**

- ↑ Most efficient cooling system
- ↓ Climate dependent
- ↓ May require auxiliary cooling when the external condition is not favourable
- ↓ Requires filtering to prevent external air contamination
- ↓ Temperature may be difficult to control (malfunction makes temperature go up quickly)

7.2 Computer Room Air Conditioning (CRAC) Based Cooling



Closed-loop system consists of two loops

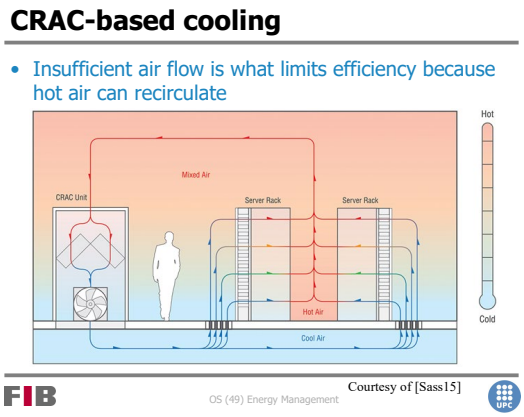
- **Air Circuit**: CRAC → Server → CRAC (Heat Exchanger)
- **Liquid Supply**: CRAC → External Heat Exchanger

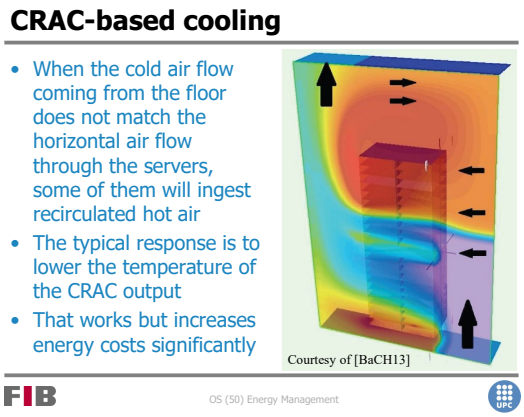
Features

- ↑ Easy to implement
- ↑ Relatively inexpensive
- ↑ Protection from external contamination
- ↓ Low operational efficiency

7.2.1 Insufficient air flow

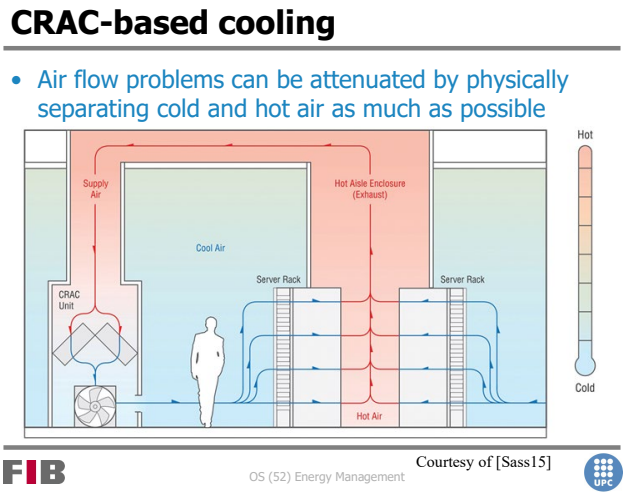
Cold air flow from the floor does not match through the servers





- limits efficiency (Hot air can circulate)
- Makes some locations to be more efficient than others
- → Lower the temperature (But increases energy costs significantly)

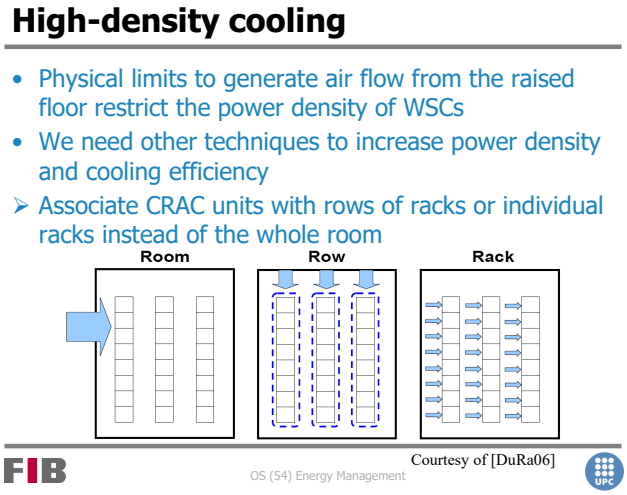
Solution: Physically separating cold and hot air



7.3 High-Density Cooling

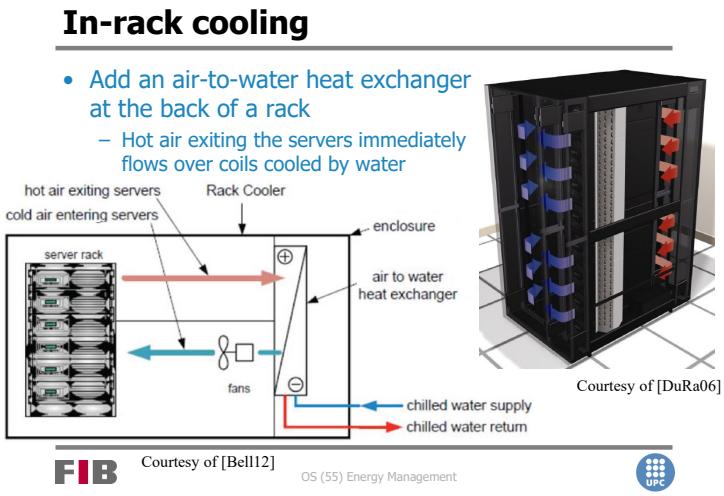
Associate CRAC unit with **rows** of racks or **individual** racks instead of the whole system

- ↑ Higher power density
- ↑ Better cooling efficiency
- ↓ Cooled water needs to be brough to each rack/row/plate → greately increasing cost and complexity
- ↓ Danger of water leakage



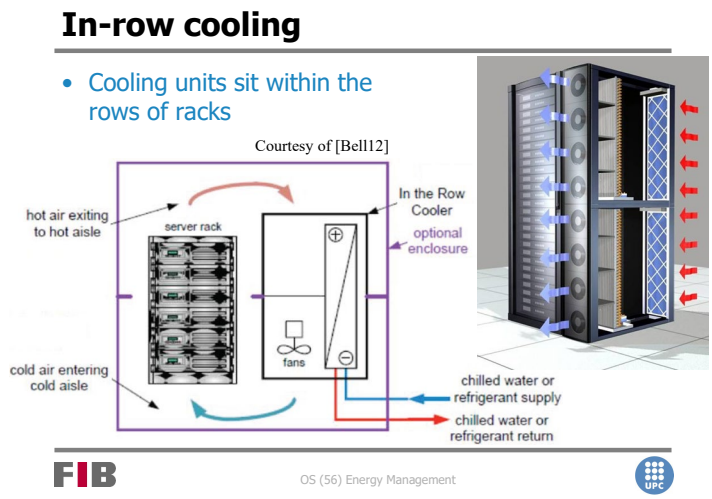
7.3.1 In-Rack Cooling

Add an air-to-water heat exchanger at the **back of the rack**



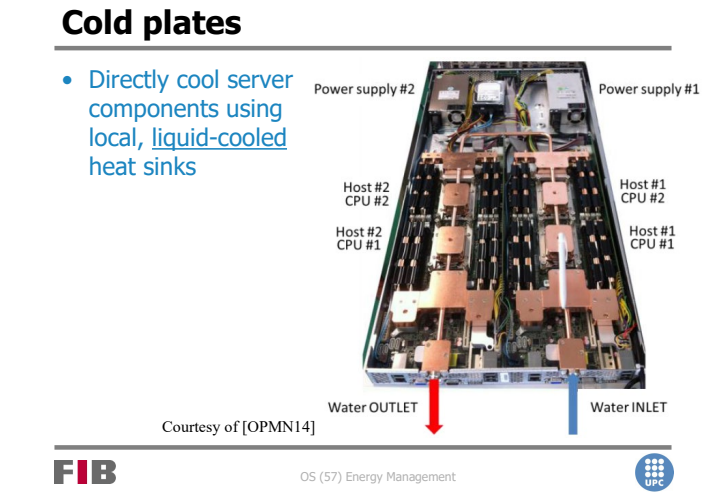
7.3.2 In-Row Cooling

Cooling unit sit **within rows of racks**



7.3.3 Cold Plates

Direct cool server components using local, liquid-cooled heat sinks



7.4 Efficiency of Cooling Subsystem

7.4.1 Hardware Perspective

- Physically separate hot and cold air
- Keep path to the cooling unit short
- Elevate temperature of the cold aisle (Equipments can work correctly in higher-than-usual temperatures)

7.4.2 Software Perspective

7.4.2.1 Workload Adapted Cooling

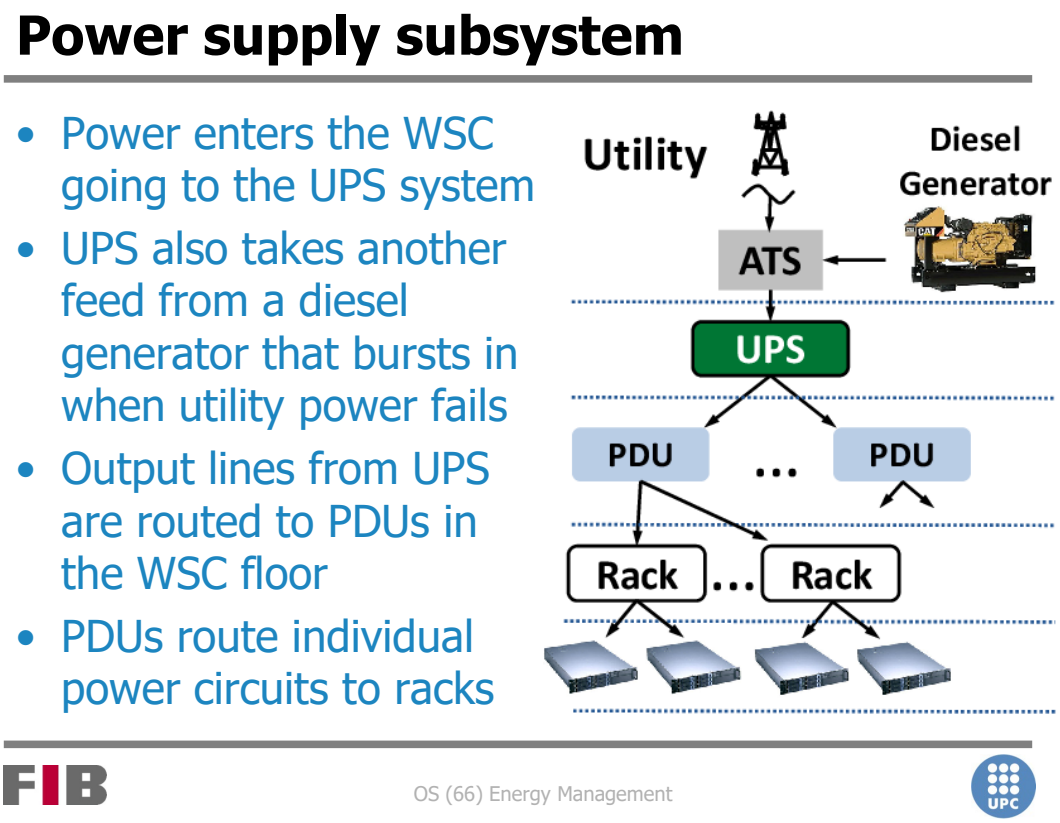
- Proactive Adapt cooling to expected workloads
  - e.g. throttle down the cooling when low IT workload is expected in the near future

7.4.2.2 Temperature-Aware Allocation

Consider the resulting temperature of hosts as a criterion to decide allocations

- Power density
  - Spread tasks to balance the load
  - Delay task start (allocate fewer resources) to smooth the load
  - Extend task execution time (DVFS) to smooth the load
- Place hot tasks in the hosts that are **easier to cool**
- Schedule according to the impact of task in the temperature

8 Power Supply Subsystem



8.1 Uninterruptible Power Supply (UPS)

8.1.1 Active Power Input Switch

- Utility grid power
- On-site power
  - Diesel generator (used when utility power fails)
  - Renewable sources (if available)

8.1.2 Energy Storage

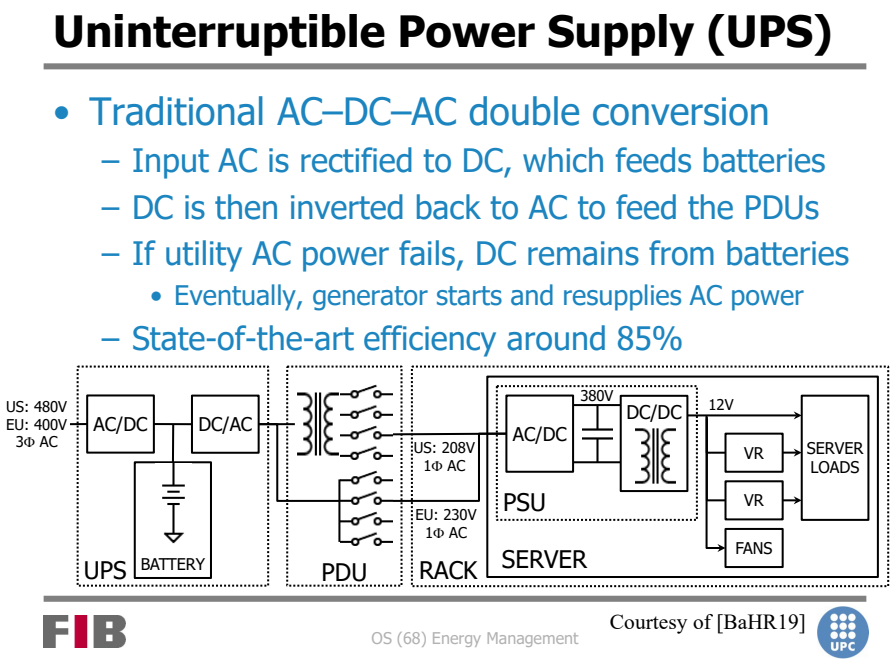
Bridge the time between **utility failure** and availability of **on-site power**

8.1.3 Remove Voltage Spikes

Remove voltage spikes through double conversion, some power efficiency is lost

8.1.3.1 Traditional Double Conversion

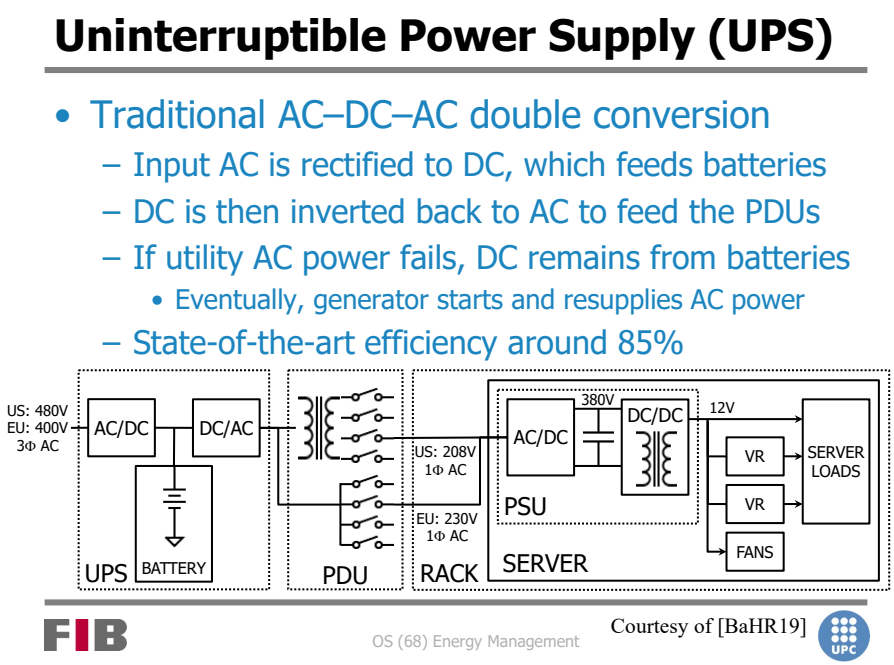
Input (AC) → Battery (DC) → Output to PDU (AC)
State-of-the-art efficiency ∼ 85%



8.1.3.2 DC System

System powered by DC, do not convert from DC back to AC
State-of-the-art efficiency: ∼ 89%

- ↑ Slightly higher power efficiency
- ↓ Higher reliability (**reduced component count**)
- ↑ Easier integration of native DC output generators
  - Solar photovoltaic, fuel cells, wind turbines
- ↓ Commercial DC equipment costs more
- ↓ DC technology in WSCs is less mature
- ↓ Limited availability of DC technicians



8.2 Power Distribution Unit (PDU)

- **Circuit Distribution**: break down input power feed from UPS into smaller circuit to distribute to individual racks
- **Safety**: **Per-Circuit Breaker** to prevent total power failure of the whole PDU or the upstream UPS

8.3 Renewable Energy

Follow-the-Renewables (Follow-the-Sun)

8.3.1 Workload Shaping

Adapt IT workload to available energy

- Use DVFS or server power state tuning to adjust **IT power demand** to time-varying **Renewable Energy Budget**
- Use **task scheduling** to postpone tasks when not enough renewable energy is available to execute

8.3.2 Efficiency of Energy Usage

- Manage short demand peaks or short-term renewable supply reductions
  - $\rightarrow$ use stored energy instead of capping the load
- Flatten the WSC load profile
  - Use less utility power when it is most expensive

8.3.3 On-site Renewable Generation

Load Following: Adapt the amount of energy generated to the expected workload

- Not adequate when workload fluctuates in short-time intervals
  - Requires time to adapt the amount of generated energy
- Combine it with workload shaping